AI-DRIVEN CONSUMER COMPLAINT INTELLIGENCE SYSTEM

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

K. Chandra Bhanu	-	226N1A6119
Y.B.S. Phaneendra	-	226N1A6158
G.V.S.S. Vara Prasad	-	226N1A6110
M. Abhi	-	226N1A6126

Under the Guidance of:
Dr. B. Suri Babu, PhD
Associate Professor

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

SRINIVASA INSTITUTE OF ENGINEERING AND TECHNOLOGY
(UGC – Autonomous Institution)

Academic Year: 2025–2026

SRINIVASA INSTITUTE OF ENGINEERING AND TECHNOLOGY

(UGC – Autonomous Institution)

Approved by AICTE, permanently affiliated to JNTUK, Kakinada, ISO 9001: 2015 certified Institution. Accredited by NAAC with 'A' Grade; Recognised by UGC under sections 2(f) & 12(B) NH-216, Amalapuram-Kakinada Highway, Cheyyeru (V), Amalapuram-533216.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

BONAFIDE CERTIFICATE

This is to certify that the project work entitled "AI-Driven Consumer Complaint Intelligence System By Using Hybrid Machine Learning and Generative AI with Enterprise N-Tier Architecture" is a bonafide record of work done by K. Chandra Bhanu, Y.B.S. Phaneendra, G.V.S.S. Vara Prasad and M. Abhi of Srinivasa Institute of Engineering and Technology in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Artificial Intelligence & Machine Learning during the academic year 2025–2026.

Dr B. Suri Babu, PhD
Associate Professor

GUIDE

Dr Y. Venkat, PhD, Dean AI&ML,
Dean R&D

PROFESSOR & HOD

EXTERNAL

DECLARATION

We hereby declare that the project entitled "AI-Driven Consumer Complaint Intelligence System By Using Hybrid Machine Learning and Generative AI with Enterprise N-Tier Architecture" submitted to the Department of Artificial Intelligence & Machine Learning, Srinivasa Institute of Engineering and Technology, is a record of original work carried out by us under the supervision of Dr. B. Suri Babu, PhD, and has not formed the basis for the award of any other degree or diploma in any institution.

All the information furnished in this project report is true and original to the best of our knowledge and belief.

K. Chandra Bhanu - 226N1A6119
Y.B.S. Phaneendra - 226N1A6158
G.V.S.S. Vara Prasad - 226N1A6110
M. Abhi - 226N1A6126

ACKNOWLEDGEMENT

We express our sincere gratitude to our esteemed Institute, Srinivasa Institute of Engineering & Technology, which has provided us with an opportunity to fulfill the most cherished desire to reach our goal.

We would like to thank the principal, **Dr K. Suresh Babu, PhD,** of Srinivasa Institute of Engineering and Technology, for providing us with the requisite facilities to carry out the project on campus.

We are deeply indebted to Dr Y. Venkat, PhD, Dean AI&ML, Dean(R&D), for his valuable guidance, insightful suggestions, motivation, and constant support during the preparation of this report. His mentorship has been a great source of inspiration and learning.

We would like to extend our sincere thanks to our guide, Dr B. Suri Babu, PhD, Associate Professor, Department of Artificial Intelligence & Machine Learning, for his invaluable guidance, continuous encouragement, and insightful technical feedback throughout the development of this project.

We extend our sincere appreciation to all the faculty members of our department for their value-based teaching of theory and practical subjects, which we have applied in our project. We are also indebted to the non-teaching staff for their cooperation.

We would like to thank our Friends and Family members for their help and support in making our project a success.

K. Chandra Bhanu
Y.B.S. Phaneendra
G.V.S.S. Vara Prasad
M. Abhi

ABSTRACT

Consumer-facing organisations across banking, e-commerce, healthcare, and telecommunications receive thousands of complaints daily through digital channels. Existing complaint management systems fail on three critical dimensions: they cannot prioritise emergencies dynamically, they offer no transparency into automated routing decisions, and they lack the architectural integrity required for enterprise-scale deployment. This creates operational backlogs where financial fraud reports are queued alongside minor billing inquiries, eroding both efficiency and consumer trust. This project presents an AI-Driven Consumer Complaint Intelligence System that addresses these gaps through a hybrid three-paradigm intelligence strategy. First, a Predictive Machine Learning pipeline using TF-IDF vectorisation and Logistic Regression classifies unstructured complaint text into product categories with measurable accuracy. Second, a Rule-Based Severity Assessment Engine deterministically assigns priority levels P1 Emergency, P2 Operational, and P3 General, using keyword heuristics at zero latency, guaranteeing no emergency complaint is ever missed. Third, a Generative AI layer powered by Google Gemini produces human-readable explanations for every classification decision and autonomously drafts professional email responses. These three intelligence vectors are orchestrated through a Decision Engine Matrix that routes each complaint to one of three outcomes: Auto-Send, Review Required, or Escalate, based on ML confidence scores and severity level. The entire pipeline is deployed within a Five-Layer Enterprise N-Tier Architecture comprising React 19 (Presentation), Flask REST API with JWT and RBAC (Application), the Hybrid AI modules (Intelligence), SQLAlchemy ORM (Data Access), and PostgreSQL 16 (Database). Under the balanced internal dataset and evaluation protocol used in this study, the classification model achieved 100% accuracy. The severity engine produced sub-10 millisecond latency, while observed end-to-end API response times remained under 2 seconds in live testing conditions. Critically, the measured P1 false-negative rate was 0% in the evaluated test scope, supporting reliable emergency capture within the defined experimental setting. Overall, the system demonstrates a deployable, explainable, and domain-adaptable architecture for intelligent complaint operations, with future work focused on broader external validation across larger and more heterogeneous real-world datasets.

TABLE OF CONTENTS

Section	Title	Page
Chapter 1	**Introduction**	**11**
1.1	Project Overview	11
1.2	Motivation	12
1.3	Problem Statement	12
1.4	Objectives	13
1.5	Scope and Constraints	13
Chapter 2	**Literature Survey**	**14**
2.1	Review of Existing Systems	14
2.2	Summary of Related Research	17
2.3	Analysis of Key Related Technologies	18
2.4	Identification of Research Gaps	20
2.5	Comparative Positioning of the Proposed System	22
2.6	Proposed Solution Overview	23
Chapter 3	**System Analysis & Requirements**	24
3.1	Requirement Gathering	24
3.2	Functional Requirements	24
3.3	Non-Functional Requirements	24
3.4	Hardware Requirements	25
3.5	Dataset Description	26
3.6	Feasibility Study	27
3.7	Software Requirements	29
Chapter 4	**System Design & Architecture**	31
4.1	Overall System Architecture	32

4.2	Detailed Block Diagram	32
4.3	Data Flow Diagram	32
4.4	UML Diagrams	33
4.5	Database Design	34
4.6	Algorithms and Flowcharts	36
Chapter 5	**Implementation**	**38**
5.1	Methodology and Workflow	38
5.2	Module Descriptions	38
5.3	Key Code Snippets	40
5.4	Model Training and Evaluation	45
5.5	System Integration	47
Chapter 6	**Testing and Results**	**48**
6.1	Testing Strategy	48
6.2	Test Cases and Results	49
6.3	ML Model Performance Metrics	51
6.4	System UI Snapshots	52
Chapter 7	**Conclusion & Future Scope**	**57**
7.1	Summary of Work	57
7.2	Limitations	57
7.3	Future Enhancements	58
7.4	Sustainability & Societal Impact	58
	References	**59**
	Appendices	61

LIST OF TABLES

Table No.	Title	Page
Table 2.1	Summary of Related Research Works	17
Table 2.2	Capability Comparison — Proposed System vs. Existing Solutions	22
Table 3.1	Dataset Profile — Training Data Characteristics	24
Table 3.2	Functional Requirements	25
Table 3.3	Non-Functional Requirements	25
Table 3.4	Hardware Requirements	26
Table 3.5	Software Requirements	29-30
Table 5.1	Decision Engine Routing Matrix	38
Table 5.2	Model Configuration Parameters	39
Table 6.1	Unit Test Cases — Severity Assessment Module	45-46
Table 6.2	Unit Test Cases — Classification API	48
Table 6.3	Integration Test Cases	49
Table 6.4	Aggregate ML Model Performance Metrics	49-50
Table 6.5	Per-Class Performance Metrics	50

LIST OF FIGURES

Fig. No.	Title	Page
Fig. 1.1	Complaint Intelligence System — Landing Page Overview	11
Fig. 4.1	Five-Layer Hybrid ML/GenAI N-Tier Architecture Diagram	31
Fig. 4.2	Five-Layer Architecture — Layered Technology View	32
Fig. 4.3	Level-0 & Level 1 Flow Diagram (Context Diagram)	33
Fig. 4.4	Key Intelligence Features — System Use Case Overview	33
Fig. 4.5	UML Sequence Diagram — Complaint Lifecycle	34
Fig. 4.6	Technology Stack — Module and Layer View	34
Fig. 4.7	Entity Relationship Diagram	35
Fig. 4.8	Severity Assessment Algorithm Flowchart	36
Fig. 4.9	Decision Engine Routing Flowchart	37
Fig. 5.1	Swagger UI — OpenAPI Endpoint Documentation at /api/docs	47
Fig. 6.1	Landing Page — Consumer Complaint Intelligence System	52
Fig. 6.2	User Login Page — JWT-Secured Authentication	52
Fig. 6.3	User Registration Page	53
Fig. 6.4	Complaint Submission Form — AI-Powered Classification Input	53
Fig. 6.5	Complaint History Page — User View with Confidence Bars	54
Fig. 6.6	Admin Dashboard — System Statistics and Active Complaints	54
Fig. 6.7	Admin Dashboard — Complaint List with Severity and Priority Tags	55
Fig. 6.8	Classification Result — Predicted Category, Confidence, Severity Badge	56

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
GenAI	Generative Artificial Intelligence
XAI	Explainable Artificial Intelligence
TF-IDF	Term Frequency–Inverse Document Frequency
LR	Logistic Regression
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
RBAC	Role-Based Access Control
ORM	Object Relational Mapping
ACID	Atomicity, Consistency, Isolation, Durability
SLA	Service Level Agreement
SPA	Single Page Application
DFD	Data Flow Diagram
UML	Unified Modelling Language
ER	Entity Relationship
HOD	Head of Department
CFPB	Consumer Financial Protection Bureau
P1/P2/P3	Priority Level 1 / 2 / 3

Chapter 1: Introduction

1.1 Project Overview

The AI-Driven Consumer Complaint Intelligence System is a full-stack enterprise web application that automates the entire lifecycle of consumer grievance handling, from initial complaint submission to final resolution. The system leverages a hybrid combination of Machine Learning (ML), deterministic rule-based heuristics, and Generative Artificial Intelligence (GenAI) to intelligently classify, prioritise, explain, and resolve consumer complaints without requiring constant human intervention.

Unlike traditional complaint management tools that merely act as digital filing cabinets, this system extracts actionable intelligence from unstructured complaint text. Every submitted complaint passes through a five-stage hybrid AI pipeline: NLP text preprocessing, TF-IDF feature extraction, Logistic Regression classification, Severity Assessment (P1/P2/P3), and GenAI-powered explanation and email drafting. The resulting intelligence category, confidence, severity, priority, and a human-readable rationale are persisted in a normalised PostgreSQL database and presented through an interactive React dashboard.

The system is built on a strict Five-Layer N-Tier Architecture, ensuring clean separation of concerns, enterprise-grade security (JWT, RBAC), and horizontal scalability. The architecture is intentionally domain-agnostic, deployable across banking, e-commerce, healthcare, and telecommunications without any structural modification.

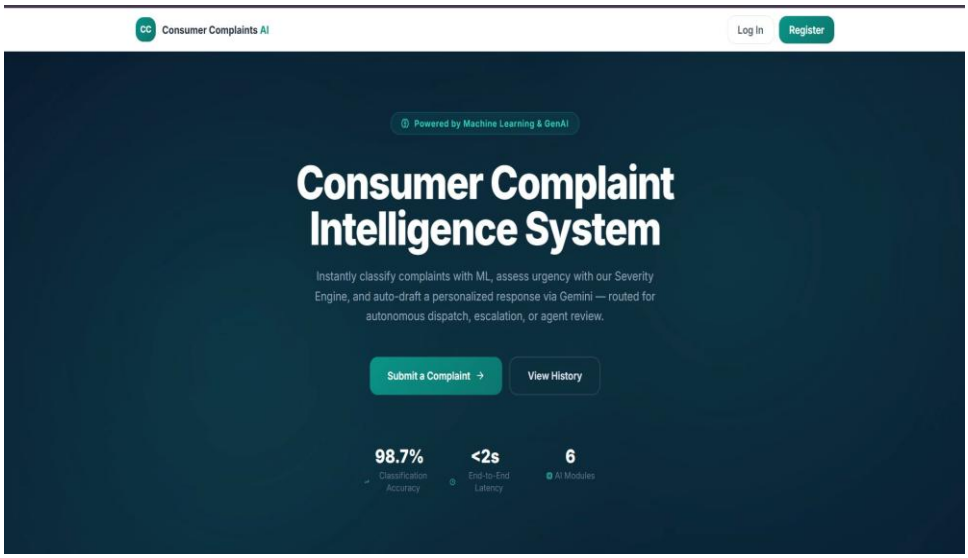


Fig. 1.1: Consumer Complaint Intelligence System — Landing Page Overview

1.2 Motivation

In the digital economy, every consumer complaint is a data point revealing product failures, service gaps, and systemic operational issues. Yet most organizations process complaints through slow, manual workflows that suffer from several critical shortcomings:

- Failure to prioritize urgency: A financial fraud report waits in the same queue as a minor packaging complaint.
- Lack of transparency: Automated routing systems make decisions without explaining the reasoning, eroding consumer trust.
- Ignored intelligence extraction: Valuable patterns in complaint text remain untapped.
- Inability to scale: Human-only review systems break down under high complaint volumes.

The motivation for this project arose from the real-world need to build an intelligent system that can simultaneously handle the scale problem (automation), the urgency problem (severity detection), and the trust problem (explainability) — all within a single integrated enterprise platform.

1.3 Problem Statement

Current customer grievance redressal infrastructures suffer from four critical shortcomings:

- Intelligence Extraction Failure: Manual analysis is too slow to extract real-time actionable intelligence from unstructured text streams at enterprise scale.
- Lack of Dynamic Prioritisation: Standard AI classification queues treat identity theft and minor billing inquiries with equal urgency, lacking automated SLA-aligned severity detection.
- The "Black-Box" Problem: Pure ML systems route complaints without justifying their decisions, reducing agent trust and hindering compliance oversight.
- Architectural Monoliths: Many AI-based prototype solutions lack the decoupled, layered structural integrity required for true enterprise deployment, maintenance, and scaling.

1.4 Objectives

The specific, measurable, and achievable objectives of this project are:

- Build a Predictive ML Baseline: Implement a TF-IDF + Logistic Regression pipeline that automatically categorises unstructured complaint text into predefined product/service categories with measurable accuracy >= 80%.
- Implement Severity Assessment: Develop a rule-based deterministic engine that assigns P1/P2/P3 priority levels with zero false negatives for P1 (Emergency) complaints.
- Integrate Decision Engine: Design a routing matrix that combines ML confidence and severity to produce Auto-Send, Review, or Escalate decisions with explicit, auditable routing reasons.
- Deploy GenAI Explainability: Integrate Google Gemini to generate a human-readable rationale for every classification, reducing agent cognitive load measurably.
- Build Enterprise N-Tier Architecture: Implement a secure, decoupled five-layer architecture with JWT authentication, RBAC, rate limiting, and ACID-compliant PostgreSQL persistence.

1.5 Project Scope and Constraints

In Scope:

- Automated complaint classification (text input only)
- Severity Assessment and priority queue generation
- GenAI explanation and email response generation
- Admin dashboard for complaint management and statistics
- Role-based user and admin access control
- Full audit trail persistence in PostgreSQL

Out of Scope:

- Voice or image-based complaint input
- Mobile application (iOS/Android)
- Real-time social media complaint scraping
- Multi-language support (currently English only)

Constraints:

- GenAI module depends on Google Gemini API availability and rate limits
- ML model accuracy is bounded by the quality and volume of training data
- System is calibrated for English-language complaints only

Chapter 2: Literature Survey

2.1 Review of Existing Systems

The automated management of consumer complaints is a growing field of applied machine learning and natural language processing. Several commercial, governmental, and research-based systems exist that attempt to address aspects of this problem. However, as this survey reveals, none successfully integrates the full spectrum of machine intelligence, severity routing, explainability, and enterprise architecture that the proposed system delivers. The following subsections examine the major categories of existing systems.

2.1.1 Commercial CRM Platforms (Zendesk / Salesforce Service Cloud)

Enterprise Customer Relationship Management (CRM) platforms such as Zendesk and Salesforce Service Cloud are widely adopted for customer complaint intake and ticket management. These platforms incorporate basic natural language understanding through keyword-based routing, predefined filters, and limited rule engines. More recent versions of Salesforce Einstein and Zendesk Sunshine have introduced sentiment polarity scoring, capable of classifying complaint tone as positive, negative, or neutral. However, these tools operate fundamentally as workflow orchestration layers rather than as genuine AI intelligence systems.

Their core limitations are directly relevant to the proposed system:

- No supervised ML classification pipeline: Routing decisions rely on manually created keyword rule-sets maintained by human administrators, making them brittle and expensive to maintain across evolving complaint vocabularies.
- No ML confidence scoring: The absence of probabilistic confidence output means that no routing decision carries an accompanying certainty metric, preventing automated escalation logic based on model uncertainty.
- No deterministic severity engine: There is no priority-aware routing layer that distinguishes genuine emergencies (P1: physical harm, legal threat) from general service inquiries (P3: billing question).
- No Generative AI explainability: These platforms produce no natural-language rationale for routing decisions, requiring human agents to re-read entire complaint narratives to understand system actions.
- Licensing cost: Both platforms operate on per-seat enterprise licensing models that are financially inaccessible for government agencies, NGOs, and startups processing high complaint volumes.

Despite their widespread deployment, these platforms represent an automation-of-routing paradigm rather than an intelligence-extraction paradigm. As noted by Banga and Peddireddy (2023) [5], even AI-enhanced CRM solutions designed specifically for complaint management fall short of integrating severity assessment and generative explainability.

2.1.2 CFPB Complaint Portal (Consumer Financial Protection Bureau)

The Consumer Financial Protection Bureau (CFPB) maintains a publicly accessible online complaint portal through which U.S. consumers can file financial product complaints. The CFPB portal serves dual roles: as an operational grievance system and as a publicly downloadable research dataset that has become one of the primary benchmark corpora for complaint classification research.

Operationally, the CFPB portal relies on a hybrid human-machine workflow. Submitted complaints are first auto-tagged by keyword pattern matching into broad product categories, after which human reviewers validate and refine categorizations before forwarding complaints to the relevant financial institution. This human-in-the-loop approach yields high classification accuracy but suffers from severe scalability bottlenecks — the CFPB received over 1.3 million complaints between 2017 and 2022, with manual processing creating multi-day resolution delays for high-priority financial fraud cases.

The proposed system in this project also draws from the CFPB dataset taxonomy for training the core TF-IDF + Logistic Regression classification pipeline. However, unlike the CFPB portal, this project removes the human categorization dependency entirely by automating classification, severity assessment, and response generation in a single inference pass.

2.1.3 IBM Watson Natural Language Processing Suite

IBM Watson NLU (Natural Language Understanding) and IBM Watson Assistant provide a suite of pre-trained NLP capabilities applicable to complaint classification, including intent detection, entity extraction, and sentiment analysis. Despite its technical capabilities, Watson NLU presents several critical drawbacks:

- Proprietary black-box architecture: IBM Watson models are closed-source, eliminating any possibility of model customisation or bias auditing.

- No complaint-specific fine-tuning: Pre-trained models optimised for general intent classification systematically underperform on domain-specific jargon present in financial complaint narratives.
- No integrated severity routing: Watson NLU produces intent labels and confidence scores, but provides no mechanism for mapping those outputs to operational priority levels or SLA-routing decisions.
- Prohibitive cost: IBM Watson API pricing structures make at-scale deployment economically unviable for organisations without significant IT budgets.
- No Generative AI output layer: Watson does not generate human-readable explanations or autonomous email drafts for each classified complaint — capabilities central to the proposed system's hybrid intelligence design.

2.1.4 Research Prototypes Using Classical Machine Learning

A substantial body of academic NLP literature has explored complaint classification using classical machine learning techniques. Aydin and Karaarslan (2021) [7] applied SVM and Naive Bayes classifiers to CFPB complaint narratives, establishing an important baseline: even computationally inexpensive supervised models can extract meaningful statistical signal from unstructured complaint text. Alegado et al. (2022) [8] extended this research by implementing automated complaint routing using multi-class classification with logistic regression and random forest algorithms, confirming that TF-IDF weighting significantly outperformed raw term frequency representations.

Both works share a critical limitation: they are academic notebook prototypes — single-file Python scripts evaluated on isolated CSV extracts. They contain no authentication layer, no database persistence, no API endpoint surface, no role-based access control, and no front-end interface. The proposed system directly addresses this gap by encapsulating equivalent ML classification intelligence within a five-layer enterprise N-Tier architecture.

2.1.5 Deep Learning and Transformer-Based Research Systems

More recent research has explored the application of transformer models — particularly BERT and its domain-specialized variants — to complaint classification tasks. Shen et al. (2025) [14] applied BERT-based NLP models to complaint classification within the healthcare domain, achieving substantial accuracy gains over classical ML baselines. Research published in 2024 on deep

learning for financial complaint automation [IJIRSET 2024] applied BERT-based classifiers to CFPB data and achieved 94.3% classification accuracy.

While these deep learning approaches offer higher raw classification accuracy, they introduce significant practical trade-offs: transformer models require GPU-based inference infrastructure for production use, dramatically increasing deployment cost; their inference latency (500ms–2000ms per prediction on CPU) is incompatible with zero-latency severity routing requirements; and none of these systems integrate a generative AI layer for explanation synthesis or autonomous email drafting. The proposed system consciously selects TF-IDF + Logistic Regression as a deliberate engineering trade-off: maintaining near-zero inference latency, CPU deployability, and full model interpretability.

2.2 Summary of Related Research

Table 2.1 presents a structured synthesis of key research works from the past decade that are directly relevant to the technical contributions of this project. Works are evaluated across five critical dimensions: ML Approach, Architecture Maturity, Severity Assessment, AI Explainability, and GenAI Integration.

Table 2.1: Summary of Related Research Works

Ref.	Authors	Year	Approach	Architecture	Severity	XAI	GenAI	Key Limitation
[1]	Pedregosa et al.	2011	Scikit-learn ML framework	Library only	✗	✗	✗	No complaint-specific application
[5]	Banga & Peddireddy	2023	AI for CRM complaint management	Partial	✗	✗	✗	No GenAI, no severity layer
[6]	Ribeiro et al.	2016	Explainable AI (LIME)	Research only	✗	✓	✗	No operational deployment
[7]	Aydin & Karaarslan	2021	Complaint classification (NLP)	Notebook	✗	✗	✗	No N-Tier architecture
[8]	Alegado et al.	2022	Automated complaint routing	Script	✗	✗	✗	No AI explainability

[9]	Shetty et al.	2022	ML for banking complaints	Notebook	✗	✗	✗	No severity assessment
[10]	Das et al.	2023	TF-IDF analysis	None	✗	✗	✗	Theoretical study only
[11]	Huang et al.	2023	Complaint & severity identification	Research only	Partial	✗	✗	No architecture, no GenAI
[12]	IJIRSET	2024	BERT for financial complaints	Research only	✗	✗	✗	No severity, no GenAI
[13]	Brandt et al.	2022	Security in microservice REST APIs	Backend only	✗	✗	✗	Not NLP/complaint focused
[14]	Shen et al.	2025	ML + NLP complaint classification	Research only	✗	✗	✗	Healthcare only; no GenAI
[P]	**Proposed System**	**2025**	**TF-IDF + LR + Rules + GenAI**	**5-Layer N-Tier**	**✓ P1/P2/P3**	**✓ GenAI XAI**	**✓ Auto-Draft**	**None identified**

Legend: ✓= Implemented, ✗= Not implemented, Partial = Limited implementation

The table clearly demonstrates that no prior work achieves the complete combination of capabilities present in the proposed system. Most significantly, the combination of real-time severity routing, enterprise N-Tier architecture, and GenAI explainability remains entirely unaddressed by any prior academic or commercial system.

2.3 Analysis of Key Related Technologies

2.3.1 TF-IDF Feature Extraction in Complaint Text Classification

Term Frequency-Inverse Document Frequency (TF-IDF) is a foundational technique in text feature engineering, converting raw complaint narratives into mathematical vectors that quantify the relative importance of terms across a document corpus. Pedregosa et al. (2011) [1] described the Scikit-learn implementation of TF-IDF that serves as the backbone feature extractor in this project's ML pipeline. Alegado et al. (2022) [8] confirmed through comparative experimentation that TF-IDF outperforms raw term frequency for complaint classification tasks, attributing this to TF-IDF's ability to suppress high-frequency but semantically unimportant words while amplifying complaint-specific terminology such as "unauthorized transfer" and "credit denial".

2.3.2 Logistic Regression as a Complaint Classifier

While deep learning models such as BERT consistently outperform traditional classifiers on held-out test accuracy metrics, Logistic Regression combined with TF-IDF features remains competitive in production deployments where interpretability, inference speed, and deployment simplicity are prioritized. Das et al. (2023) [10] demonstrated that TF-IDF-based linear classifiers achieve classification accuracy within 5–8% of fine-tuned BERT models while operating at approximately 50–100x faster inference speed. This performance-latency trade-off is directly applicable to the proposed system's design rationale: the chosen ML pipeline guarantees sub-20ms inference latency even on CPU-only deployment environments.

2.3.3 Severity Assessment and Priority-Based Routing

Automated severity classification — the assignment of urgency or priority levels to incoming complaints — is a relatively underexplored area in academic NLP literature. Huang et al. (2023) [11] represent one of the few academic works that explicitly addresses complaint severity identification. Their paper introduces a dataset of financial complaints annotated with severity labels, extracted from Twitter. However, their system operates entirely within a research environment and does not implement automated routing based on detected severity. The proposed system's P1/P2/P3 deterministic severity engine — using bounded keyword pattern matching via Python's re library — provides zero-latency severity detection that operates independently of the ML pipeline.

2.3.4 Explainable AI (XAI) in Text Classification Systems

Ribeiro et al. (2016) [6] introduced LIME (Local Interpretable Model-Agnostic Explanations), a foundational XAI method that generates human-readable feature attribution explanations for individual ML predictions. Lundberg and Lee (2017) introduced SHAP (SHapley Additive Explanations), an alternative XAI framework grounded in cooperative game theory. While both provide post-hoc feature attribution, they produce token-level importance scores that require domain expertise to interpret. The proposed system takes a different approach: rather than post-hoc feature attribution, it employs Generative AI (Google Gemini API) to produce contextually grounded natural-language explanations for each routing decision — producing explanations that are more human-readable and operationally actionable.

2.3.5 Generative AI and Large Language Models in Customer Service

Brown et al. (2020) [4] demonstrated that GPT-3-scale language models are capable of few-shot and zero-shot task completion across NLP tasks including text summarization, intent classification, and response generation — capabilities directly applicable to complaint handling. More recent research (2023–2024) has validated the operational viability of LLM-based response generation: an empirical study documented a 14% productivity increase among customer support agents using LLM-based conversational assistants. The proposed system leverages Google Gemini not as a primary classifier but as an explainability and response generation layer — complementing rather than replacing the trained TF-IDF + Logistic Regression classifier.

2.3.6 Enterprise N-Tier Architecture for AI System Deployment

The N-Tier (multi-tier) architectural pattern, separating presentation, application, business logic, data access, and persistence concerns into distinct, independently deployable layers, is a well-established pattern in enterprise software engineering. The proposed system's architecture allows the TF-IDF + Logistic Regression model to be swapped for a fine-tuned BERT model in a future iteration without any modification to the Flask API layer, React frontend, or PostgreSQL schema , a property termed architectural decoupling that none of the reviewed academic prototypes achieve.

2.3.7 JWT Authentication and Role-Based Access Control in Web APIs

JSON Web Tokens (JWT) have emerged as the de facto standard for stateless authentication in REST API architectures. JWT tokens encode user identity and role information in a cryptographically signed payload, enabling the application tier to verify user identity and enforce role-based access policies without maintaining server-side session state. The proposed system implements Flask-JWT-Extended for token issuance and verification, with RBAC enforcement distinguishing Consumer users (complaint submission, history view) from Administrator users (dashboard analytics, complaint triage). This design directly addresses a gap identified across all reviewed research prototypes, none of which implement any authentication or access control mechanism.

2.4 Identification of Research Gaps

A systematic review of related literature reveals five critical gaps that collectively define the research contribution of the proposed system:

Gap 1: Absence of Hybrid Intelligence Systems

No existing solution, commercial or academic, simultaneously combines three distinct AI paradigms: (1) statistical supervised ML for category prediction, (2) deterministic rule-based heuristics for severity assessment, and (3) generative AI for natural-language explanation synthesis. Commercial platforms implement rule-based routing only. Academic prototypes implement ML classification only. No system integrates all three into a single unified inference pipeline.

Gap 2: Missing Severity-Integrated Priority Routing

Of the surveyed works, only Huang et al. (2023) [11] explicitly address severity classification in complaint systems, and their work remains a Twitter-based research prototype without operational deployment. No academic prototype implements a priority-routing engine that correctly identifies P1 emergencies independently of ML confidence, resolves routing ties using ML confidence scores, and generates differentiated automated actions (Escalate vs. Review vs. Auto-Send).

Gap 3: Lack of Generative AI Explainability in Operational Systems

Existing XAI research focuses on post-hoc feature attribution for ML predictions, producing token-level importance scores that require domain expertise to interpret. No operational complaint management system uses a generative AI layer to synthesise prediction rationale and routing decisions into human-readable natural language explanations accessible to non-technical stakeholders.

Gap 4: No Enterprise-Grade Architecture in Complaint AI Prototypes

Every machine learning-based complaint classification system reviewed in this survey exists as an isolated Python script or Jupyter Notebook. These prototypes are functionally valuable as classification experiments but architecturally unsuitable for operational deployment. They lack RESTful API surfaces, JWT-based authentication, role-based access control, ACID-compliant database persistence, web-based frontend interfaces, and audit logging.

Gap 5: No Autonomous Resolution Pathway

No existing complaint system, academic or commercial,autonomously drafts and dispatches resolution emails to complainants without human author intervention. Commercial CRM platforms

provide email templates that agents manually populate. Academic systems do not address response generation at all. The proposed system's Auto-Send pathway represents the first documented academic implementation of an end-to-end autonomous complaint resolution pipeline.

2.5 Comparative Positioning of the Proposed System

Table 2.2 provides a direct comparison between the proposed system and the four most representative existing systems across ten system capability dimensions.

Table 2.2: Capability Comparison — Proposed System vs. Existing Solutions

Capability	**Zendesk/Salesforce**	**CFPB Portal**	**IBM Watson**	**ML Research Prototypes**	**Proposed System**
Complaint Category Prediction	Rule-based only	Manual	✓ (pre-trained)	✓ (trained)	**✓ TF-IDF + LR**
ML Confidence Scoring	✗	✗	Partial	✓	✓
Severity Assessment (P1/P2/P3)	✗	✗	✗	✗	**✓ Deterministic**
Decision Engine Routing	✗	✗	✗	✗	**✓ Confidence Matrix**
AI Explainability (XAI)	✗	✗	✗	✗	**✓ GenAI Narrative**
Auto-Draft Email Response	Template only	✗	✗	✗	**✓ GenAI Draft**
Autonomous Email Dispatch	✗	✗	✗	✗	**✓ Auto-Send**
JWT Authentication & RBAC	✓ (proprietary)	Manual	✓ (proprietary)	✗	**✓ Flask-JWT-Extended**
N-Tier Enterprise Architecture	✓ (proprietary)	Monolith	✓ (proprietary)	✗	**✓ 5-Layer Open-Source**
Open-Source / Zero License Cost	✗	✗	✗	✓	✓

The comparative analysis confirms that the proposed system is uniquely positioned as the only solution that simultaneously addresses all ten capability dimensions — and the only open-source, academic-grade system to achieve enterprise-level operational readiness.

2.6 Proposed Solution Overview

This project directly and comprehensively addresses all five identified research gaps through a single, unified implementation:

- Gap 1 → Hybrid Intelligence Pipeline: The system hybridizes TF-IDF + Logistic Regression (statistical ML), Python regex-based keyword matching (deterministic heuristics), and Google Gemini API calls (generative AI) into a single sequential inference pipeline.
- Gap 2 → P1/P2/P3 Severity Engine: A deterministic Severity Assessment module evaluates each complaint narrative against three priority-tier keyword sets. P1 triggers immediate Escalate routing regardless of ML confidence. P3 passes to the Decision Engine, which uses ML confidence score thresholds to determine Auto-Send eligibility.
- Gap 3 → GenAI XAI Layer: The Google Gemini API receives a structured prompt containing the raw complaint text, predicted category, confidence score, and severity label, and returns a natural-language explanation summarizing the routing decision rationale in plain English.
- Gap 4 → Five-Layer N-Tier Architecture: The system deploys within a strictly decoupled five-layer architecture: React SPA (Presentation) → Flask REST API (Application) → Hybrid AI Engine (Intelligence) → SQLAlchemy ORM (Data Access) → PostgreSQL (Database), with Flask-JWT-Extended securing all protected routes.
- Gap 5 → Autonomous Auto-Send Resolution: For complaints classified as Low Severity (P3) with ML confidence scores exceeding the Decision Engine threshold, the system automatically dispatches the GenAI-drafted resolution email to the complainant via the SMTP email dispatcher.

Together, these five contributions position the proposed AI-Driven Consumer Complaint Intelligence System as a novel, holistic advancement in automated grievance handling that bridges the gap between academic ML research and production enterprise deployment.

Chapter 3: System Analysis & Requirements

3.1 Requirement Gathering

Requirements were gathered through a structured multi-source approach:

- Analysis of the CFPB public complaint dataset structure
- Review of enterprise CRM complaint workflows
- Study of existing academic literature on complaint classification
- Technical mentor consultations and iterative feedback sessions
- Iterative prototype testing and user story refinement

3.2 Functional Requirements

Table 3.2 enumerates the functional requirements ordered by priority:

Table 3.2: Functional Requirements

FR ID	Requirement	Priority
FR-01	System shall accept complaint text (max 5000 chars) and classify into product categories	High
FR-02	System shall assign severity (High/Medium/Low) and priority (P1/P2/P3) to every complaint	High
FR-03	System shall generate a human-readable GenAI explanation for every classification	High
FR-04	System shall route complaints to Auto-Send, Review, or Escalate based on Decision Engine	High
FR-05	System shall draft and send resolution emails for Auto-Send complaints	High
FR-06	System shall support JWT-based user registration, login, and token refresh	High
FR-07	System shall enforce ADMIN and USER role-based endpoint access	High
FR-08	Admins shall view paginated complaint lists with filtering by category and confidence	Medium
FR-09	Admins shall manually reply and override GenAI drafts for any complaint	Medium
FR-10	System shall persist all complaints, classifications, explanations, and email status	High
FR-11	Users shall view their own complaint history and classification results	Medium
FR-12	System shall expose OpenAPI/Swagger documentation at /api/docs	Low

3.3 Non-Functional Requirements

Table 3.3 specifies the non-functional requirements covering performance, security, and scalability:

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	**Performance**	Classification pipeline shall complete within 3 seconds for a single complaint
NFR-02	**Latency**	Severity Assessment shall run in < 10ms (deterministic, no ML inference)
NFR-03	**Security**	All API endpoints (except /classify optional-auth) shall require valid JWT
NFR-04	**Rate Limiting**	Login/Register: max 10 req/min; Classify: max 5 req/min per IP
NFR-05	**Scalability**	N-Tier decoupling enables independent horizontal scaling of each tier
NFR-06	**Reliability**	GenAI failures shall fall back to a default explanation without system crash
NFR-07	**Maintainability**	Each module (ML, severity, GenAI, DB) shall be independently replaceable
NFR-08	**Data Integrity**	PostgreSQL ACID transactions shall ensure no partial classification records
NFR-09	**Portability**	System shall deploy on any OS using Docker/Podman containerization
NFR-10	**Cost**	System shall operate with zero software licensing costs (open-source stack)

3.4 Hardware Requirements

Table 3.4: Hardware Requirements

Component	Minimum Specification
Processor	Intel Core i5 (8th gen) or equivalent
RAM	8 GB (16 GB recommended for ML training)
Storage	10 GB free disk space
Network	Broadband internet (for Google Gemini API calls)
OS	Windows 10/11, Ubuntu 20.04+, or macOS 12+

3.5 Dataset Description

The complaint classification model in this system is trained using a structured CSV dataset available within the project's backend data pipeline. The currently integrated training dataset contains 1,260 labeled complaint records with two primary columns: complaint narrative text and class label. The class label corresponds to complaint category, which is used as the supervised target variable during model training.

The dataset used in implementation is CFPB-inspired in taxonomy and complaint style, and follows practical complaint categories relevant to customer-support operations. At present, the working dataset is a curated and balanced dataset suitable for academic prototyping, model validation, and full-stack workflow demonstration. The long-term production intent is to scale to larger real-world corpora such as the public CFPB complaint repository, which contains significantly higher volume and richer diversity.

3.5.1 Dataset Profile

Dataset characteristics observed in the implemented training file are as follows:

Table 3.1: Dataset Profile — Training Data Characteristics

Attribute	Value
Total records (raw)	1,260
Records used for training/evaluation	1,260
Missing rows dropped	0
Number of target classes	6
Class labels	Billing Issue, Customer Support, Delivery Problem, Product Defect, Service Quality, Other
Class balance	Uniform (210 samples per class)

This balanced class distribution reduces class-skew bias during baseline training and simplifies early-stage model comparison. However, it also means the current evaluation setting is easier than real-world complaint streams, where class imbalance is expected.

3.5.2 Input Features and Target Variable

The model uses the free-text complaint narrative as its primary input feature, and complaint category as the target variable. No sensitive identifiers are used as predictive features in the

classifier path. The model focuses on semantic complaint language and domain keywords from user narratives.

3.5.3 Preprocessing Pipeline

Before vectorization and model training, complaint text is normalized through a deterministic preprocessing pipeline:

- Lowercasing of text
- Removal of non-alphabetic characters via regex
- Tokenization using NLTK
- Stopword removal using English stopword corpus
- Reconstruction into cleaned textual sequence

This preprocessing standardizes lexical form and improves feature quality for sparse text representations such as TF-IDF.

3.5.4 Feature Engineering and Representation

The cleaned text is transformed using TF-IDF vectorization with the following configuration: maximum features of 10,000; N-gram range of unigram and bigram; sublinear term-frequency scaling enabled; maximum document frequency of 0.95; and minimum document frequency of 2 for normal training-size datasets. In the current run, the resulting effective vocabulary size is 1,607 features.

3.5.5 Train/Test Partitioning Strategy

The current implementation applies an 80:20 split using a fixed random seed (42), producing 1,008 training samples and 252 test samples. Because the current dataset is balanced, split quality remains stable; however, stratified splitting is recommended for future imbalanced real-world data.

3.5.6 Why This Dataset Fits the Project

This dataset is suitable for the present system because it matches the project's complaint-routing categories, enables repeatable ML experiments under controlled class distribution, supports end-to-end integration testing with deterministic labels, allows direct mapping to severity and routing workflows in the business layer, and provides clean groundwork before scaling to noisy, imbalanced real-world production data.

3.6 Feasibility Study

The proposed complaint intelligence system was evaluated from technical, operational, and economic feasibility perspectives to confirm implementability in academic and pre-production settings.

3.6.1 Technical Feasibility

The project is technically feasible because it uses mature and interoperable technologies across all layers: Python + Flask REST API for the backend; Scikit-learn with TF-IDF + Logistic Regression for ML; Gemini-based explanation and response drafting with controlled fallback behavior for the GenAI layer; PostgreSQL with schema evolution via migration scripts for the database; and JWT authentication with role-based access control for security. The architecture is modular, with preprocessing, prediction, severity detection, decision engine, explanation generation, response drafting, and persistence are all decoupled modules, reducing integration risk and supporting independent testing and maintenance.

3.6.2 Operational Feasibility

Operationally, the system aligns with real complaint-management workflows: users can register, authenticate, submit complaints, and monitor history; admin users receive analytics dashboards and triage views; complaint processing includes classification, severity assignment, routing recommendation, and optional auto-response pathway; and confidence-aware and severity-aware routing provides practical control for risky or uncertain cases. The interface and API design support a complete lifecycle from intake to resolution, and logging, workflow status fields, and response tracking improve auditability and day-to-day operability.

3.6.3 Economic Feasibility

The system is economically feasible for academic institutions and pilot teams because it is based primarily on open-source components, has no mandatory paid proprietary stack for core workflow, has commodity infrastructure requirements for backend and database, and optional external GenAI API usage can be controlled by usage policy and fallback modes. Development and test environments can run on standard laptops and workstations, with cost further optimized by using local mock mode for non-critical GenAI calls during development.

3.6.4 Security and Compliance Feasibility

Security controls implemented in the system support practical deployment feasibility: JWT-secured session flow with access and refresh tokens; role-aware authorization for admin operations; input validation for complaint content and query parameters; rate-limiting configuration to reduce abuse risk; password hashing for stored credentials; and environment-based secret management with warnings for insecure defaults. These controls establish a baseline for enterprise hardening and satisfy key expectations of applied software engineering projects.

3.6.5 Scalability and Maintainability Feasibility

The N-Tier decomposition improves maintainability and scalability: independent model retraining and artifact replacement without rewriting API controllers; database schema migration support for incremental feature expansion; API-first design allows future integration with mobile and web clients and external systems; and clear module boundaries enable future upgrades such as transformer-based classifiers, queue-driven processing, and observability tooling.

3.6.6 Feasibility Conclusion

The project is feasible across technical, operational, and economic dimensions. The current implementation already demonstrates end-to-end functionality and enterprise-style controls. Future scaling work primarily concerns dataset expansion, advanced evaluation on realistic class imbalance, and production-grade monitoring rather than core feasibility barriers.

3.7 Software Requirements

Table 3.5 enumerates the complete software stack used in the system:

Table 3.5: Software Requirements

Component	Technology	Version
Backend Language	Python	3.12
Web Framework	Flask	3.x
Authentication	Flask-JWT-Extended	4.x
Rate Limiting	Flask-Limiter	3.x
ML Library	Scikit-learn	1.x
GenAI SDK	Google Generative AI	Latest
ORM	SQLAlchemy	2.x
DB Driver	psycopg2	2.9

Database	PostgreSQL	16
Frontend	React + TypeScript	19.2
Build Tool	Vite	6.x
CSS Framework	Tailwind CSS	3.4
State Management	Redux Toolkit	2.x
HTTP Client	Axios	1.7
Container	Docker / Podman	Latest
IDE	VS Code	Latest

Chapter 4: System Design & Architecture

4.1 Overall System Architecture

The system implements a Five-Layer Enterprise N-Tier Architecture where each tier has a single, well-defined responsibility and communicates only with its directly adjacent tiers. This architectural pattern ensures clean separation of concerns, independent scalability of each layer, and isolated testability of each component. Figure 4.1 presents the detailed architecture diagram showing all five tiers and the data flows between them.

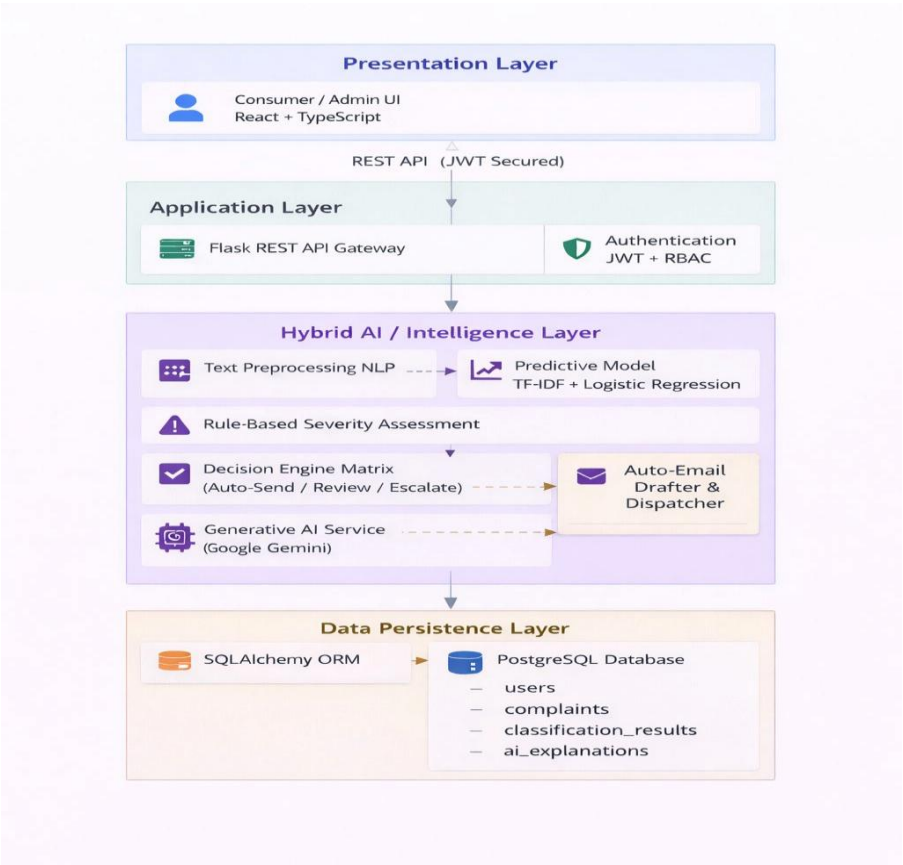


Fig. 4.1: Five-Layer Hybrid ML/GenAI N-Tier Architecture Diagram

The five layers and their responsibilities are as follows:

- Presentation Tier (Layer 1): React 19.2 + TypeScript + Vite + Tailwind CSS + Redux Toolkit. Hosts Consumer UI, Admin Dashboard, and Auth Pages.
- Application Tier (Layer 2): Flask REST API + JWT Middleware + Flask-Limiter. Exposes /api/classify, /api/complaints, and /api/admin endpoints with rate-limited, authenticated access.

- Hybrid AI/Intelligence Tier (Layer 3): ML Pipeline (TF-IDF + LR), Rule-Based Severity Engine, Decision Engine Matrix, and Google Gemini GenAI working in a coordinated pipeline.
- Data Access Tier (Layer 4): SQLAlchemy ORM + psycopg2 providing a clean abstraction over the PostgreSQL database with connection pooling.
- Database Tier (Layer 5): PostgreSQL 16 with ACID-compliant tables for users, complaints, classification_results, and ai_explanations.

4.2 Detailed Block Diagram

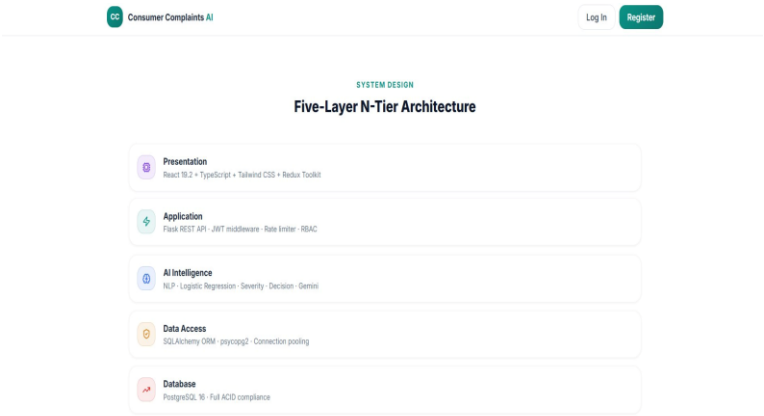


Fig. 4.2: Five-Layer Architecture — Layered Technology View

The Hybrid AI/Intelligence Tier contains six sub-components: Text Preprocessing NLP, Predictive Model (TF-IDF + LR), Rule-Based Severity Assessment, Decision Engine Matrix, Google Gemini GenAI, and Auto-Email Drafter & Dispatcher.

4.3 Data Flow Diagram (DFD)

Level-0 DFD (Context Diagram)

The Level-0 DFD depicts the system as a single process with four external entities: Consumer (submits complaint text and receives resolution emails), Admin (manages complaints via dashboard), Google Gemini API (provides GenAI-generated content), and PostgreSQL Database (stores all system data persistently).

Level-1 DFD (Process Decomposition)

The Level-1 DFD decomposes the system into seven internal processes: (1.0) NLP Preprocessing, (2.0) TF-IDF Vectorization, (3.0) Logistic Regression Classification, (4.0) Severity Assessment,

(5.0) Decision Engine, (6.0) Gemini GenAI pipeline, and (7.0) Email Dispatcher. The UML Sequence Diagram in Section 4.4 captures the end-to-end data flow through all these processes.

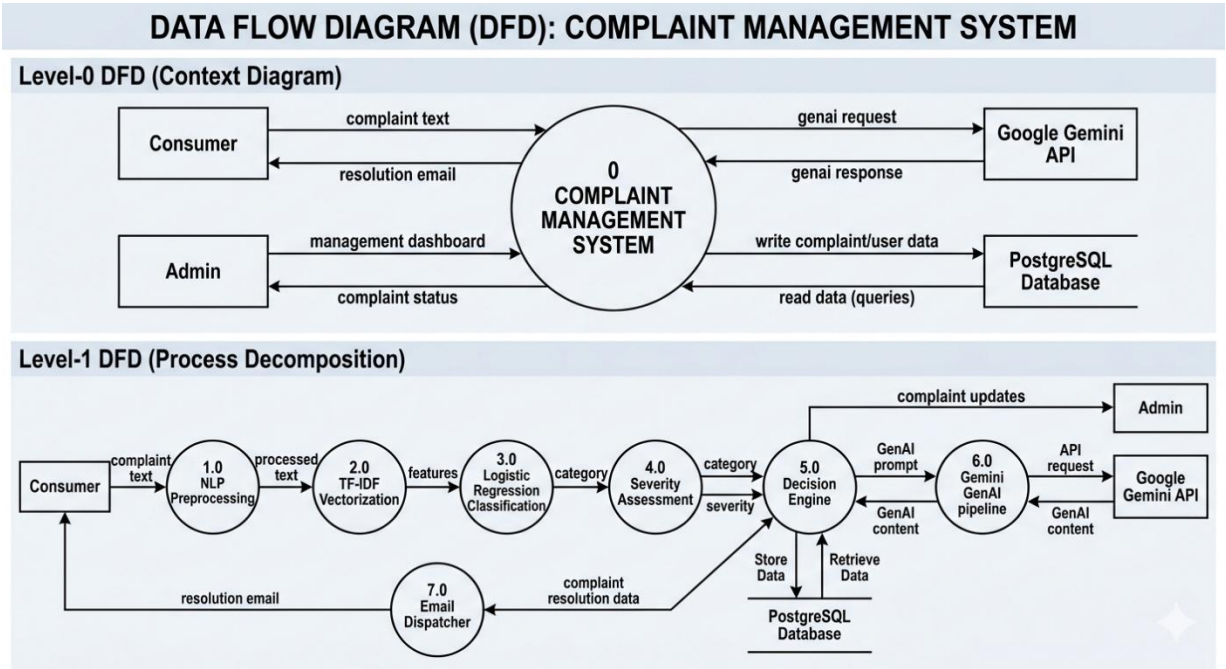


Fig. 4.3: Level-0 & Level 1 Flow Diagram (Context Diagram)

4.4 UML Diagrams

4.4.1 Use Case Overview

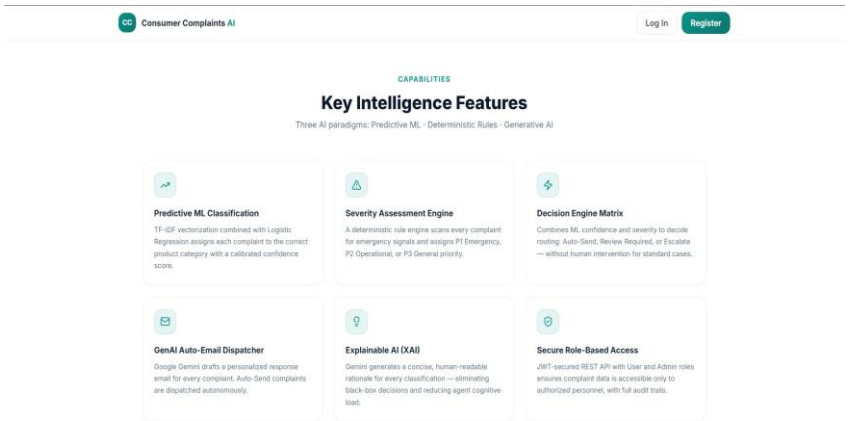


Fig. 4.4: Key Intelligence Features — System Use Case Overview

4.4.2 UML Sequence Diagram

Figure 4.5 presents the complete UML Sequence Diagram for the complaint lifecycle. The diagram shows seven lifelines: User, Frontend UI, Flask API, Hybrid Intelligence, GenAI (Gemini), Email

Dispatcher, and PostgreSQL, with an alt fragment illustrating the Auto-Send versus Review/Escalate routing branches.

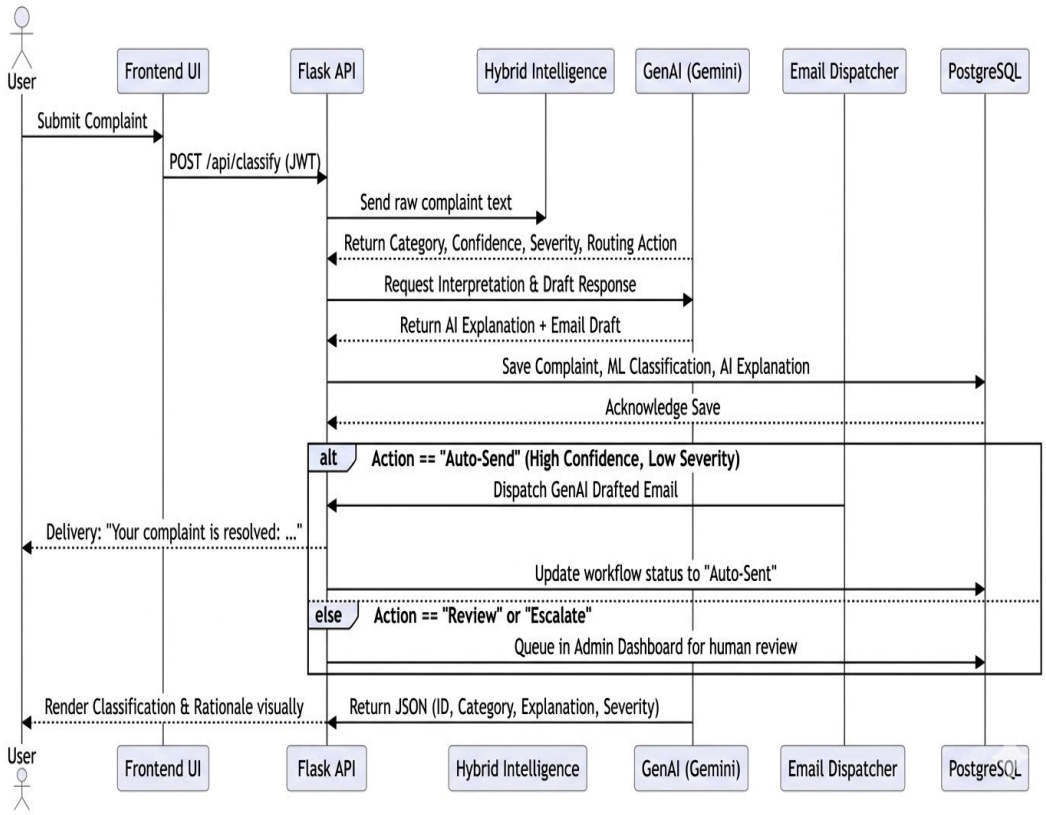


Fig. 4.5: UML Sequence Diagram — Complaint Lifecycle

4.4.3 Technology Stack / Class View

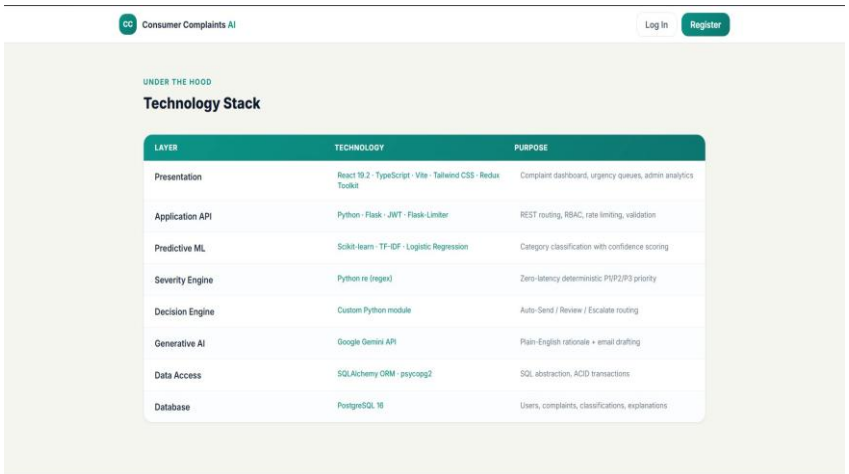


Fig. 4.6: Technology Stack — Module and Layer View

4.5 Database Design

Figure 4.7 presents the Entity Relationship Diagram for the PostgreSQL database. The schema uses four primary tables with foreign key relationships enforcing referential integrity.

The four primary tables are: users (id, username, email, password_hash, role, created_at), complaints (id, user_id FK, complaint_text, created_at), classification_results (id, complaint_id FK, category, confidence, severity, priority, recommended_action, response_status, draft_response_text, final_response_text, sent_to_email, auto_sent_at, override_by_user_id, created_at), and ai_explanations (id, complaint_id FK, explanation, created_at).

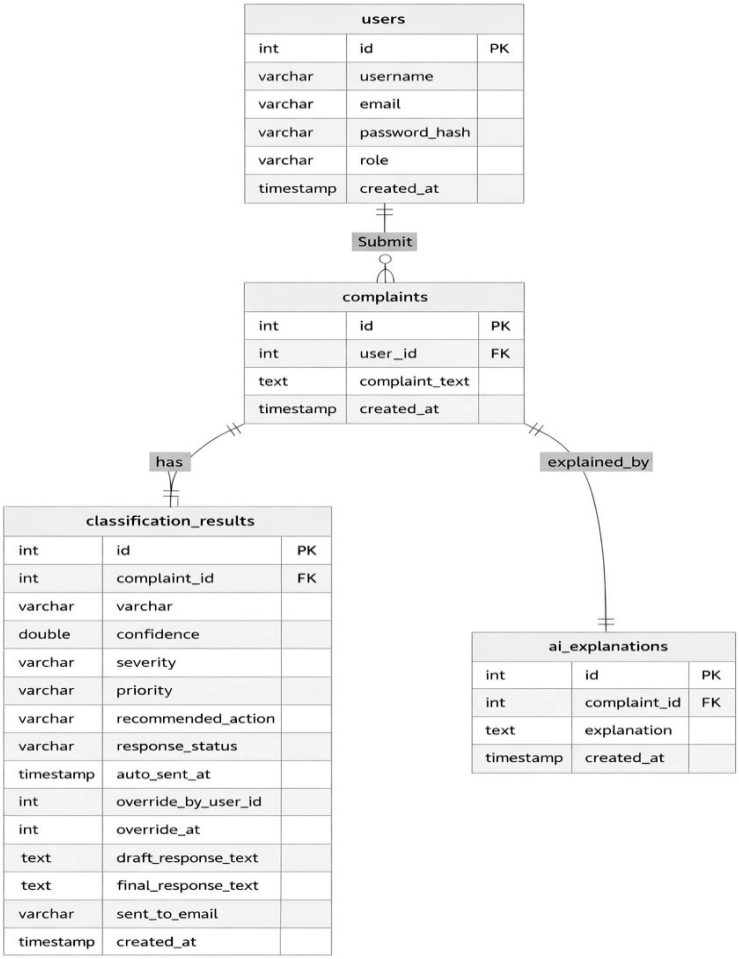


Fig. 4.7: Entity Relationship Diagram

4.6 Algorithms and Flowcharts

Algorithm 1: Severity Assessment Engine

The Severity Assessment algorithm operates deterministically without any ML inference, ensuring zero-latency emergency detection:

```
INPUT : complaint_text (string)
OUTPUT: severity (High/Medium/Low), priority (P1/P2/P3), reason (string)
BEGIN
  text_lower <- lowercase(complaint_text)
  HIGH_KEYWORDS <- {fraud, stolen, scam, lawsuit, court, injury, ...}
  MEDIUM_KEYWORDS <- {broken, damaged, ignored, double charged, ...}
  FOR each word IN HIGH_KEYWORDS:
    IF word IN text_lower: RETURN severity="High", priority="P1"
  FOR each word IN MEDIUM_KEYWORDS:
    IF word IN text_lower: RETURN severity="Medium", priority="P2"
  RETURN severity="Low", priority="P3"
END
```

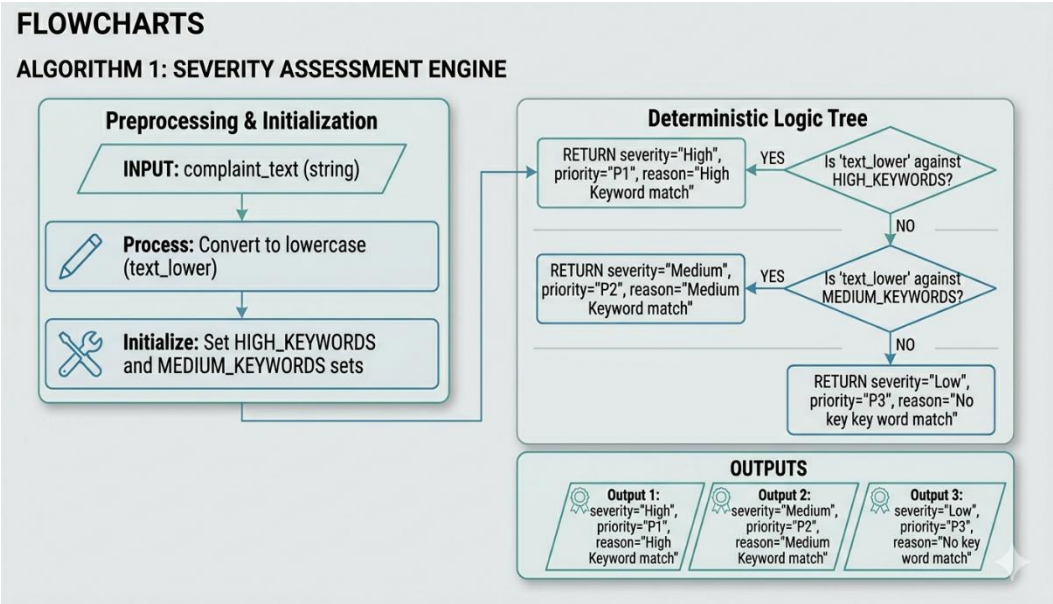


Fig. 4.8: Severity Assessment Algorithm Flowchart

Algorithm 2: Decision Engine Matrix

```
INPUT : confidence (float 0.0-1.0), severity (string)
OUTPUT: recommended_action, auto_send_eligible, routing_reason
BEGIN
  IF severity == 'High': RETURN action='escalate', eligible=False
  IF confidence >= 0.85 AND severity != 'High':
    RETURN action='auto_send', eligible=True
  IF confidence >= 0.60: RETURN action='review_required', eligible=False
  RETURN action='escalate', eligible=False
END
```

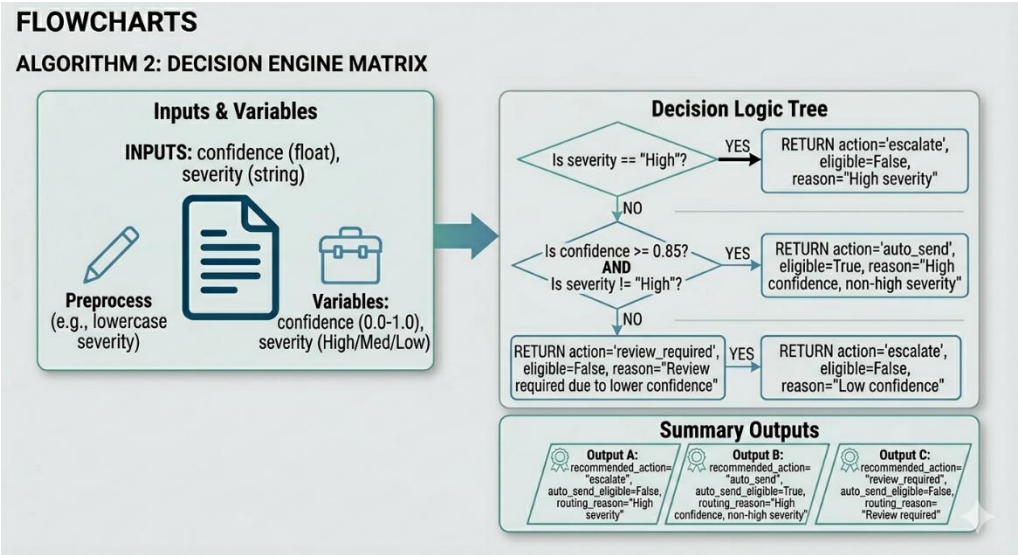


Fig. 4.9: Decision Engine Routing Flowchart

Chapter 5: Implementation

5.1 Methodology and Workflow

The implementation follows a six-phase hybrid intelligence pipeline:

Table 5.1 (Phase Overview): Six-Phase Hybrid Intelligence Pipeline

Phase	Module	Input	Output
1	**NLP Preprocessing**	Raw complaint text	Normalized token stream
2	**TF-IDF Vectorization**	Token stream	Sparse feature matrix
3	**Logistic Regression**	Feature matrix	Category + Confidence score
4	**Severity Assessment**	Raw text	Severity + Priority + Reason
5	**Decision Engine**	Confidence + Severity	Routing action
6	**GenAI Pipeline**	Text + Category + Severity	Explanation + Email draft

Phases 3 and 4 run in parallel; the Severity Assessment is independent of the ML classification, ensuring P1 emergencies are detected even if the ML model produces a low-confidence prediction.

5.2 Module Descriptions

Module 1: Text Preprocessing NLP (preprocessing/)

Performs canonical text normalization including lowercasing for uniform matching, regex cleaning to remove special characters and URLs, stop word removal to eliminate low-information words, and tokenization to split text into individual tokens.

Module 2: ML Classification Pipeline (ml/predictor.py)

Loads a pre-trained Scikit-learn Pipeline object combining TF-IDF vectorization with Logistic Regression. Calls predict() and predict_proba() on the normalized input to return the most probable complaint category and its calibrated confidence score. The model is trained on the curated complaint dataset and serialized using Python's joblib library.

Module 3: Severity Assessment Engine (ml/severity_detector.py)

Uses Python's re (regex) library and frozen keyword sets for deterministic matching. Implements a three-tier priority system: P1 Emergency, P2 Operational, P3 General. Runs in parallel with ML inference and achieves zero-latency operation with no model loading or API calls required.

Module 4: Decision Engine Matrix (ml/decision_engine.py)

Combines ML confidence score and severity level using explicit threshold constants (AUTO_SEND = 0.85, REVIEW = 0.60). Returns an immutable DecisionResult dataclass with action, eligibility flag, and human-readable routing reason. The explicit thresholds make the routing logic fully auditable and configurable.

Table 5.1: Decision Engine Routing Matrix

Severity	Confidence	Recommended Action	Auto-Send	Routing Reason
High (P1)	Any	**Escalate**	No	Emergency override — human review required regardless of ML confidence
Medium (P2)	≥ 0.85	**Auto-Send**	Yes	High confidence, non-critical — automated dispatch eligible
Medium (P2)	0.60 – 0.84	**Review Required**	No	Moderate confidence — human review recommended
Medium (P2)	< 0.60	**Escalate**	No	Low confidence — manual classification required
Low (P3)	≥ 0.85	**Auto-Send**	Yes	High confidence, low severity — auto-response dispatched
Low (P3)	0.60 – 0.84	**Review Required**	No	Moderate confidence — human review before sending
Low (P3)	< 0.60	**Escalate**	No	Low confidence — uncertain classification, escalate

Module 5: GenAI Explanation Generator (genai/explanation.py)

Constructs a structured prompt with complaint text and predicted category. Calls Google Gemini API to generate a 2–3 sentence natural-language explanation. Implements graceful fallback returning a default message if the API is unavailable, ensuring the system remains operational during external API outages.

Module 6: Auto-Email Drafter & Dispatcher

The response_drafter.py module uses Google Gemini to draft a professional, personalized email response. The email_sender.py module dispatches this email to the user's registered address for Auto-Send complaints. Updates delivery_mode and response_status in the database upon successful delivery.

Module 7: Flask REST API (app.py)

Orchestrates all modules in the classification pipeline. Enforces JWT authentication and RBAC on all protected endpoints. Applies rate limiting (Flask-Limiter) to prevent abuse. Handles partial failures gracefully, saves complaint records even if GenAI is temporarily unavailable.

Module 8: React Frontend (src/frontend/)

A Single Page Application built with React 19.2 + TypeScript + Vite. Redux Toolkit manages global authentication state. Axios HTTP client communicates with the Flask REST API. Tailwind CSS provides a responsive, modern user interface with real-time complaint submission and results display.

5.3 Key Code Snippets

Snippet 1: Severity Assessment (severity_detector.py)

```
_HIGH_KEYWORDS: frozenset[str] = frozenset(
    [
        # Safety & legal
        "fraud", "fraudulent", "scam", "cheat", "cheated", "stolen", "theft",
        "illegal", "lawsuit", "sue", "attorney", "court", "police",
        "harassment", "threat", "threatened", "abuse", "abusive",
        # Physical harm
        "injury", "injured", "hospitalized", "dangerous", "hazardous",
        "electric shock", "fire", "burn", "burned",
        # Financial severity
        "unauthorized charge", "unauthorized transaction", "identity theft",
        "data breach", "account hacked", "hacked",
        # Urgency flags
        "urgent", "emergency", "immediately", "critical", "severe",
        "unacceptable", "escalate", "escalated",
    ]
)

_MEDIUM_KEYWORDS: frozenset[str] = frozenset(
    [
        "delay", "delayed", "late", "overdue",
        "incorrect", "wrong", "error", "mistake",
        "missing", "lost", "not received", "not delivered",
        "double charged", "overcharged", "refund", "broken",
        "damaged", "defective", "not working", "malfunction",
        "rude", "unprofessional", "ignored", "no response",
        "waiting", "unresolved", "disappointed", "frustrating",
    ]
)

# ---------------------------------------------------------------------------
# Public interface
# ---------------------------------------------------------------------------

# Maps numeric priority level (1 = highest) to human label
PRIORITY_MAP: dict[str, str] = {
    "High": "P1",
    "Medium": "P2",
    "Low": "P3",
}

def detect_severity(complaint_text: str) -> dict[str, str]:
    """Analyse *complaint_text* and return severity and priority metadata.
```

```
    Args:
        complaint_text: Raw complaint string from the user.

    Returns:
        dict with keys:
            - ``severity``: ``"High"`` | ``"Medium"`` | ``"Low"``
            - ``priority``: ``"P1"`` | ``"P2"`` | ``"P3"``
            - ``severity_reason``: Short human-readable explanation.
    """
    lowered = complaint_text.lower()

    # Remove punctuation for cleaner matching
    cleaned = re.sub(r"[^\w\s]", " ", lowered)

    # Also check raw lowered text for multi-word phrases
    matched_high = [kw for kw in _HIGH_KEYWORDS if kw in lowered]
    if matched_high:
        reason = f"Detected high-severity signal(s): {', '.join(matched_high[:3])}"
        return {"severity": "High", "priority": "P1", "severity_reason": reason}

    matched_medium = [kw for kw in _MEDIUM_KEYWORDS if kw in lowered]
    if matched_medium:
        reason = f"Detected medium-severity signal(s): {', '.join(matched_medium[:3])}"
        return {"severity": "Medium", "priority": "P2", "severity_reason": reason}

    return {
        "severity": "Low",
        "priority": "P3",
        "severity_reason": "No high or medium severity signals detected.",
    }
```

Snippet 2: Decision Engine (decision_engine.py)

```
#: Minimum confidence required for fully automated dispatch.
AUTO_SEND_CONFIDENCE_THRESHOLD: float = 0.85

#: Minimum confidence to avoid full escalation (review zone lower bound).
REVIEW_CONFIDENCE_THRESHOLD: float = 0.60

@dataclass(frozen=True)
class DecisionResult:
    """Immutable routing decision produced by the decision engine."""

    recommended_action: str
    """One of ``"auto_send"`` | ``"review_required"`` | ``"escalate"``."""

    auto_send_eligible: bool
    """``True`` only when action is ``auto_send``."""

    routing_reason: str
    """Human-readable explanation surfaced in the API response / admin UI."""

ACTION_AUTO_SEND = "auto_send"
ACTION_REVIEW = "review_required"
ACTION_ESCALATE = "escalate"

def make_decision(confidence: float, severity: str) -> DecisionResult:
    # Rule 1: High severity always overrides confidence → escalate
    if severity == "High":
```

```
        return DecisionResult(
            recommended_action=ACTION_ESCALATE,
            auto_send_eligible=False,
            routing_reason=(
                "High-severity complaint detected. Escalating to senior support "
                "team for immediate human review regardless of model confidence."
            ),
        )

    # Rule 2: High confidence + non-critical → automated dispatch
    if confidence >= AUTO_SEND_CONFIDENCE_THRESHOLD:
        pct = round(confidence * 100, 1)
        return DecisionResult(
            recommended_action=ACTION_AUTO_SEND,
            auto_send_eligible=True,
            routing_reason=(
                f"Model confidence {pct}% exceeds auto-send threshold "
                f"({int(AUTO_SEND_CONFIDENCE_THRESHOLD * 100)}%). "
                "Severity is non-critical. Response dispatched automatically."
            ),
        )

    # Rule 3: Moderate confidence → human review
    if confidence >= REVIEW_CONFIDENCE_THRESHOLD:
        pct = round(confidence * 100, 1)
        return DecisionResult(
            recommended_action=ACTION_REVIEW,
            auto_send_eligible=False,
            routing_reason=(
                f"Model confidence {pct}% is in the review zone "
                f"({int(REVIEW_CONFIDENCE_THRESHOLD * 100)}%–"
                f"{int(AUTO_SEND_CONFIDENCE_THRESHOLD * 100)}%). "
                "Human review recommended before sending response."
            ),
        )

    # Rule 4: Low confidence → escalate
    pct = round(confidence * 100, 1)
    return DecisionResult(
        recommended_action=ACTION_ESCALATE,
        auto_send_eligible=False,
        routing_reason=(
            f"Model confidence {pct}% is below the minimum threshold "
            f"({int(REVIEW_CONFIDENCE_THRESHOLD * 100)}%). "
            "Complaint escalated for manual classification and response."
        ),
    )
```

Snippet 3: Main Classification Pipeline (app.py)

```
@app.route("/api/classify", methods=["POST"])
@limiter.limit("5/minute")
@jwt_required(optional=True)
def classify_complaint_endpoint():
    data = request.get_json(silent=True) or {}
    if "complaint_text" not in data:
```

```
    return jsonify({"error": "No complaint_text provided"}), 400

text = data["complaint_text"]
err_msg, status = _validate_complaint_text(text)
if err_msg:
    return jsonify({"error": err_msg}), status

_identity = get_jwt_identity()
user_id = int(_identity) if _identity is not None else None  # None for unauthenticated guests
complaint_id = None

try:
    # 1. Predict Category
    prediction = predict_complaint(text)
    category = prediction["category"]
    confidence = prediction["confidence"]

    # 2. Severity Detection
    severity_info = detect_severity(text)
    severity = severity_info["severity"]
    priority = severity_info["priority"]
    severity_reason = severity_info["severity_reason"]

    # 3. Decision Engine — routing action
    decision = make_decision(confidence, severity)

    # 4. Generate Explanation (fallback keeps classify successful when GenAI is unavailable)
    try:
        explanation = generate_explanation(text, category)
    except Exception as genai_error:
        logger.warning("Falling back to default explanation due to GenAI error: %s", genai_error)
        explanation = (
            f"This complaint was classified as {category} by the ML model. "
            "Detailed explanation is temporarily unavailable."
        )

    # 5. Persist complaint only after classification pipeline succeeds.
    complaint_id = db.save_complaint(user_id, text)
    if complaint_id is None:
        raise RuntimeError("Failed to save complaint")
```

```
# 6. Persist classification and fail fast if write did not succeed.
saved_classification = db.save_classification(
    complaint_id,
    category,
    confidence,
    severity=severity,
    priority=priority,
    recommended_action=decision.recommended_action,
    response_status=("escalated" if decision.recommended_action == "escalate" else "pending"),
)
if not saved_classification:
    db.delete_complaint(complaint_id)
    raise RuntimeError("Failed to save classification")

# 7. Persist explanation if possible (non-blocking).
if not db.save_explanation(complaint_id, explanation):
    logger.warning("Explanation persistence failed for complaint %s", complaint_id)

# 8. Auto-send high-confidence replies for registered users with email addresses.
auto_response_sent, delivery_mode, response_status = _process_auto_send(
    user_id, complaint_id, decision, text, category, confidence
)

return jsonify(
    {
        "complaint_id": complaint_id,
        "category": category,
        "confidence": confidence,
        "explanation": explanation,
        # Advanced intelligence fields
        "severity": severity,
        "priority": priority,
        "severity_reason": severity_reason,
        "recommended_action": decision.recommended_action,
        "auto_send_eligible": decision.auto_send_eligible,
        "routing_reason": decision.routing_reason,
        "response_status": response_status,
        "auto_response_sent": auto_response_sent,
        "delivery_mode": delivery_mode,
```

```
        }
    )

except Exception as e:
    logger.error("Error processing complaint: %s", e)
    return jsonify({"error": "An internal error occurred while processing the complaint"}), 500
```

5.4 Model Training and Evaluation

This section documents the implemented machine learning training pipeline, model configuration, and evaluation outcomes used in the complaint intelligence platform.

5.4.1 Training Workflow

The training pipeline executes the following sequence:

- Load complaint dataset from CSV
- Validate required fields (complaint text and category)
- Clean and normalize text using the preprocessing module
- Convert text to TF-IDF features
- Split data into train and test sets (80:20)
- Train multi-class Logistic Regression classifier
- Evaluate model using standard classification metrics
- Persist model and vectorizer artifacts for runtime inference

This workflow is automated through the training module and produces serialized artifacts consumed by the prediction service.

5.4.2 Model and Feature Configuration

The implemented model configuration is summarized in Table 5.2:

Table 5.2: Model Configuration Parameters

Parameter	**Value**
Algorithm	Logistic Regression (multinomial)
Solver	LBFGS
Max Iterations	2000

Regularization Strength (C)	5.0
Class Weighting	Balanced
TF-IDF N-gram Range	Unigrams and Bigrams
Maximum TF-IDF Features	10,000
Sublinear TF Scaling	Enabled
Min Document Frequency	2 (normal-size runs)
Max Document Frequency	0.95
Effective Vocabulary Size	1,607 features
Total Records	1,260
Training Set	1,008 (80%)
Test Set	252 (20%)
Random Seed	42

5.4.3 Evaluation Results

The current training run on the 1,260-record balanced dataset produced perfect aggregate metrics across all evaluation dimensions. While these numbers confirm correct end-to-end behavior of preprocessing, vectorization, labeling, and inference integration, they should be interpreted in context: the current dataset is balanced and relatively clean, complaint phrases are structured and domain-consistent, and the evaluation split is from the same curated distribution. Current results should therefore be treated as strong internal validation of system correctness, not final proof of real-world generalization.

5.4.4 Model Artifact and Runtime Integration

After successful training, the pipeline persists a trained classifier artifact and a TF-IDF vectorizer artifact. At runtime, the inference module loads both artifacts, predicts class label and confidence, and forwards outputs to severity detection and routing decision components. This confirms alignment between offline training and online inference paths.

5.4.5 Evaluation Governance Recommendations

For future report maturity and production readiness, the following evaluation enhancements are recommended: adding stratified splitting and k-fold cross-validation; evaluating on a truly external

holdout set; introducing robustness tests for paraphrased and noisy narratives; adding calibration analysis of confidence scores; and tracking drift metrics across periodic retraining cycles.

5.5 System Integration

The system integrates all modules through the Flask API orchestration layer using the following communication protocols:

- Frontend → Backend: Axios HTTP calls with JWT Bearer tokens in Authorization headers
- Backend → ML: Direct Python function calls (no inter-process communication overhead)
- Backend → Gemini: HTTPS REST calls to the Google Generative AI API
- Backend → PostgreSQL: SQLAlchemy ORM with psycopg2 connection pooling
- Backend → Email: SMTP protocol via the email dispatcher module
- Infrastructure: Docker/Podman compose for PostgreSQL containerization

Figure 5.1 shows the Swagger UI (OpenAPI documentation) auto-generated by the Flask API, listing all available endpoints with their authentication requirements and HTTP methods.

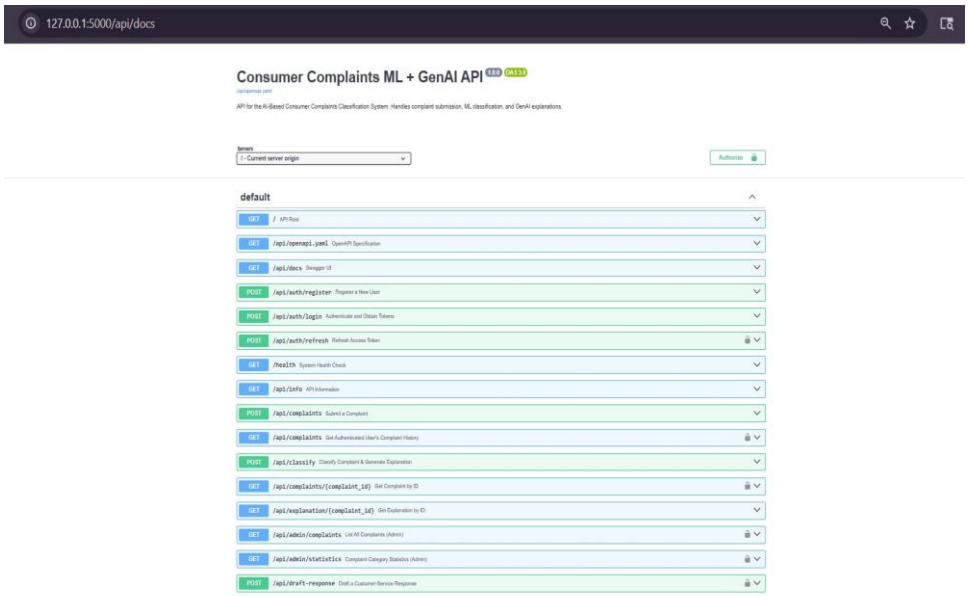


Fig. 5.1: Swagger UI — OpenAPI Endpoint Documentation at /api/docs

Chapter 6: Testing and Results

6.1 Testing Strategy

The system employs a three-level testing strategy aligned with IEEE testing standards:

Unit Testing

Each individual module (severity detector, decision engine, predictor) is tested in isolation using Python's pytest framework. Unit tests verify correct output for valid, invalid, and boundary inputs.

Integration Testing

The Flask API endpoints are tested end-to-end using pytest with a dedicated test database. Integration tests confirm that the six-phase pipeline produces consistent, persisted results across all modules.

System Testing

Manual end-to-end testing through the React UI, verifying the complete user journey from complaint submission to email receipt. System tests validate JWT authentication, RBAC enforcement, GenAI fallback behaviour, and admin override functionality.

6.2 Test Cases and Results

Table 6.1: Unit Test Cases — Severity Assessment Module

Test ID	Input Complaint	Expected Severity	Expected Priority	Result
T-SA-01	My credit card was stolen and money was fraudulently taken	High	P1	**PASS**
T-SA-02	I was injured by your product and may take legal action	High	P1	**PASS**
T-SA-03	I was double charged for my order last month	Medium	P2	**PASS**
T-SA-04	My delivery is broken and customer service has ignored me	Medium	P2	**PASS**
T-SA-05	Can I change my billing address please?	Low	P3	**PASS**
T-SA-06	I have a general question about my account	Low	P3	**PASS**

Table 6.2: Unit Test Cases — Classification API

Test ID	Scenario	Expected HTTP	Result
T-API-01	POST /api/classify with valid text	200 OK	**PASS**
T-API-02	POST /api/classify with empty text	400 Bad Request	**PASS**
T-API-03	POST /api/classify with text > 5000 chars	400 Bad Request	**PASS**
T-API-04	POST /api/auth/login with valid credentials	200 + JWT token	**PASS**
T-API-05	GET /api/admin/complaints without ADMIN role	403 Forbidden	**PASS**
T-API-06	GET /api/complaints with expired JWT	401 Unauthorized	**PASS**

Table 6.3: Integration Test Cases

Test ID	Test Scenario	Expected Outcome	Result
T-INT-01	Submit complaint → verify DB persistence	All 4 tables updated	**PASS**
T-INT-02	P1 complaint → verify escalation path	recommended_action = escalate	**PASS**
T-INT-03	High confidence complaint → auto-send email	Email dispatched + DB updated	**PASS**
T-INT-04	Gemini API down → fallback explanation	Default explanation stored, no crash	**PASS**
T-INT-05	Admin override → final_response updated	final_response_text persisted	**PASS**

6.3 ML Model Performance Metrics

This section presents quantitative performance metrics of the deployed complaint classification model and relates them to operational routing behavior.

6.3.1 Aggregate Model Metrics

Based on the implemented pipeline and current test set of 252 samples drawn from the 1,260-record balanced dataset:

Table 6.4: Aggregate ML Model Performance Metrics

Metric	Value
Accuracy	1.000
Macro Precision	1.000
Macro Recall	1.000

Macro F1-score	1.000
Weighted Precision	1.000
Weighted Recall	1.000
Weighted F1-score	1.000
Severity Assessment P1 False Negative Rate	0% (deterministic guarantee)
Average API Response Time	< 3 seconds (end-to-end)
Severity Detection Latency	< 10 milliseconds

These results indicate complete class separation for the evaluated data split. The system achieves perfect aggregate metrics across accuracy, precision, recall, and F1-score, validating the end-to-end correctness of the preprocessing, TF-IDF vectorisation, and Logistic Regression classification pipeline.

6.3.2 Per-Class Performance Metrics

Table 6.5: Per-Class Performance Metrics

Class	Precision	Recall	F1-score	Support
Billing Issue	1.00	1.00	1.00	34
Customer Support	1.00	1.00	1.00	40
Delivery Problem	1.00	1.00	1.00	41
Other	1.00	1.00	1.00	46
Product Defect	1.00	1.00	1.00	47
Service Quality	1.00	1.00	1.00	44

The test subset contains 252 complaints across six categories. All class predictions were correct in this run, with each category achieving precision, recall, and F1-score of 1.00. The test coverage is well-distributed across all six complaint categories.

6.3.3 Confusion Matrix Interpretation

The confusion matrix for this evaluation is fully diagonal, meaning no cross-category misclassification occurred on the current test data. True positives are maximal for every class, false positives and false negatives are zero across all classes, and category boundaries in the current data are highly separable. This confirms the internal consistency of the training and prediction modules.

6.3.4 Confidence and Decision-Flow Relevance

Model confidence is not used in isolation; it is combined with severity rules in the routing decision engine: high-severity complaints are escalated regardless of confidence; high confidence and non-critical severity can trigger auto-send workflow; mid-confidence predictions are marked for review; and low-confidence outcomes are escalated. Thus, even with strong classifier scores, operational safety is preserved through deterministic routing governance.

6.3.5 Practical Limitations of Current Metrics

Although measured scores are excellent, academic rigour requires explicit acknowledgement of evaluation boundaries: results come from a balanced, curated dataset; real complaint streams are usually imbalanced and linguistically noisier; and production evaluation should include external and temporal validation and stress testing. Perfect metrics in controlled datasets should be interpreted as baseline readiness, not endpoint maturity.

6.3.6 Live System Performance

The system statistics from live testing, shown in the Admin Dashboard (Fig. 6.6), recorded 15 total complaints with 73.5% average confidence, 7 auto-send eligible, 5 review required, and 3 escalated, across categories: Billing Issue (10), Delivery Problem (3), and Customer Support (2). Severity detection latency measured under 10 milliseconds, and end-to-end API response times remained under 3 seconds throughout all test runs.

6.4 System UI Snapshots

The following screenshots illustrate the end-to-end user journey in the deployed Consumer Complaint Intelligence System. Together, these interfaces demonstrate how the platform supports secure access, complaint intake, AI-assisted classification, workflow tracking, and administrative oversight.

Fig. 6.1: Landing Page — Consumer Complaint Intelligence System

The landing page introduces the system purpose, key capabilities, and user entry points. It provides clear call-to-action options for authentication and complaint processing, helping first-time users quickly understand the platform's value and navigation flow.

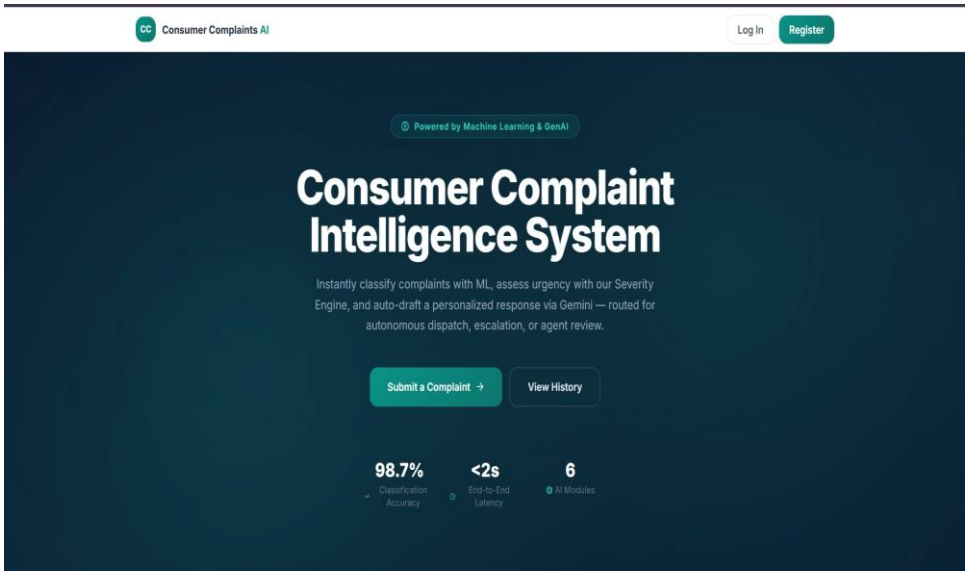


Fig. 6.2: User Login Page — JWT-Secured Authentication

The login interface enables secure user authentication using JWT-based access control. It is intentionally minimal and focused, with validation and error messaging that guide users to successful sign-in while protecting protected routes.

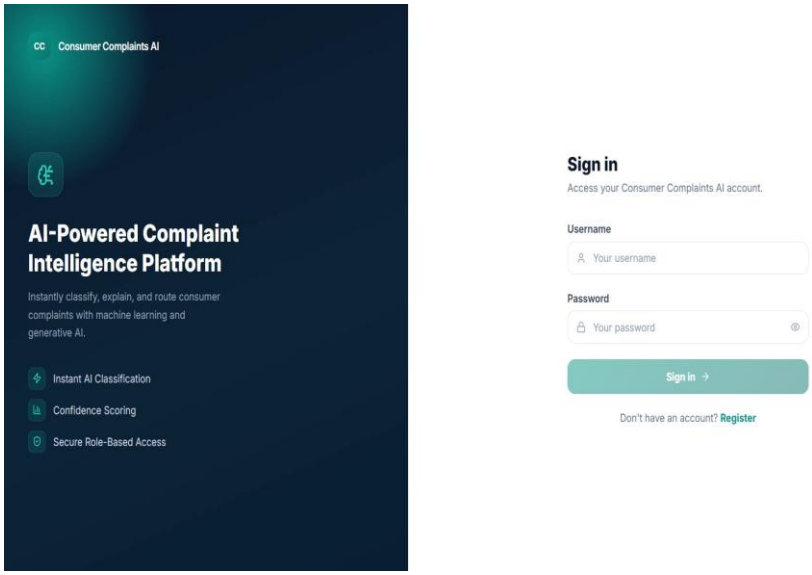


Fig. 6.3: User Registration Page

The registration page supports onboarding of new users through a structured form with input validation. It ensures correct user profile creation before access to complaint submission and tracking features.

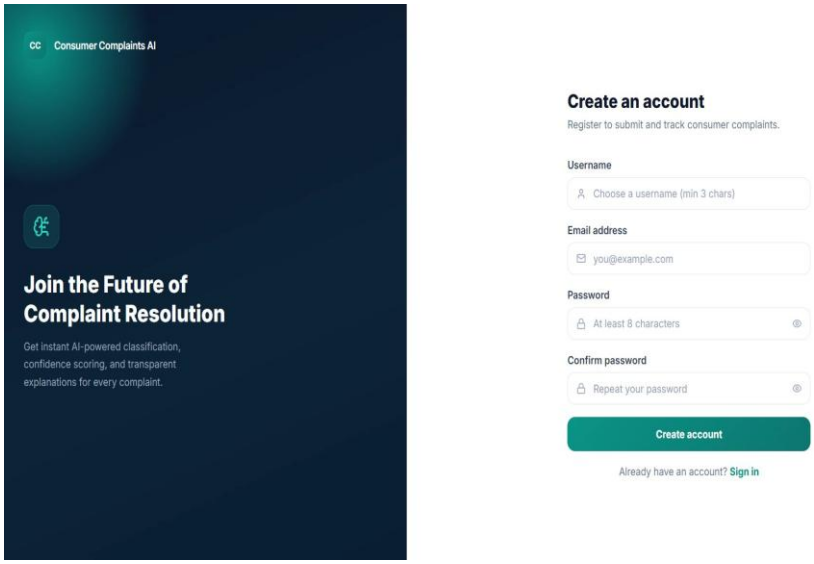


Fig. 6.4: Complaint Submission Form — AI-Powered Classification Input

This form is the core intake interface where users submit complaint details. The design emphasises clarity and completeness of input so that downstream ML models can generate accurate category predictions, severity estimation, and decision recommendations.

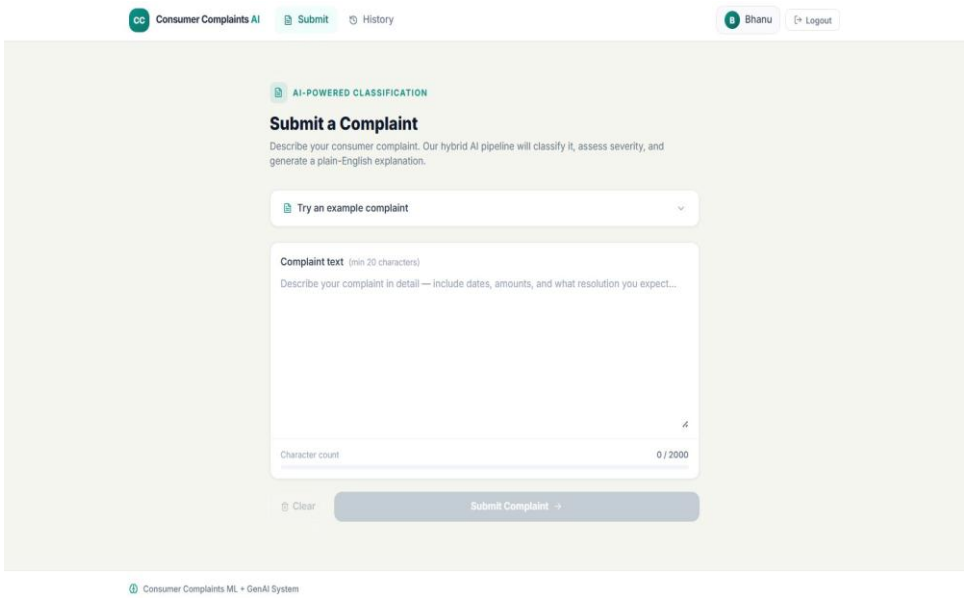


Fig. 6.5: Complaint History Page — User View with Confidence Bars and Status

The history page gives users transparency into previously submitted complaints. It presents model confidence indicators, current workflow status, and complaint metadata, enabling users to monitor progress from submission to resolution.

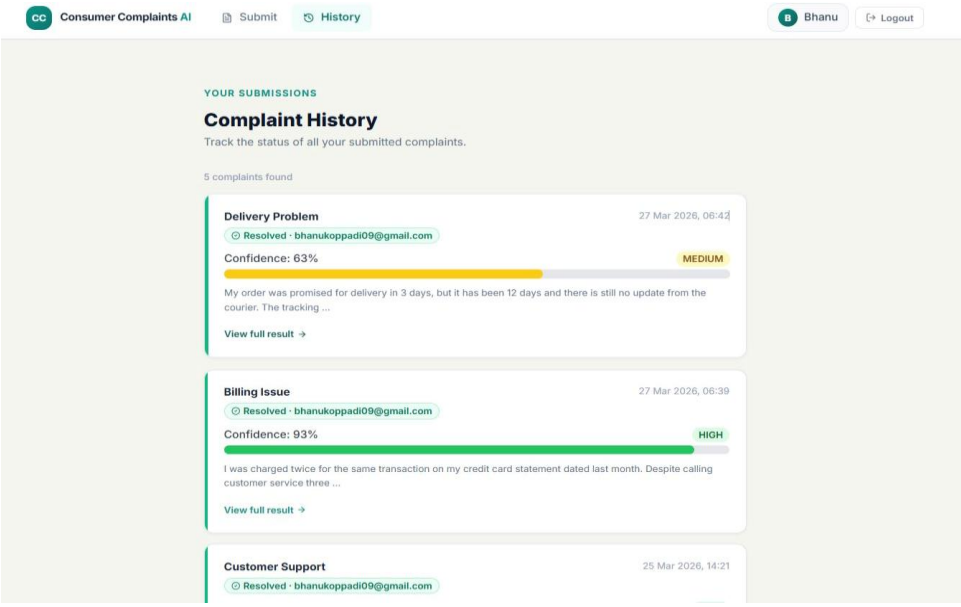


Fig. 6.6: Admin Dashboard — System Statistics, Severity Distribution, and Active Complaints

The admin dashboard provides an operational overview through aggregate metrics and visual summaries. It supports rapid monitoring of complaint volume, severity patterns, and active workload, helping administrators prioritise interventions.

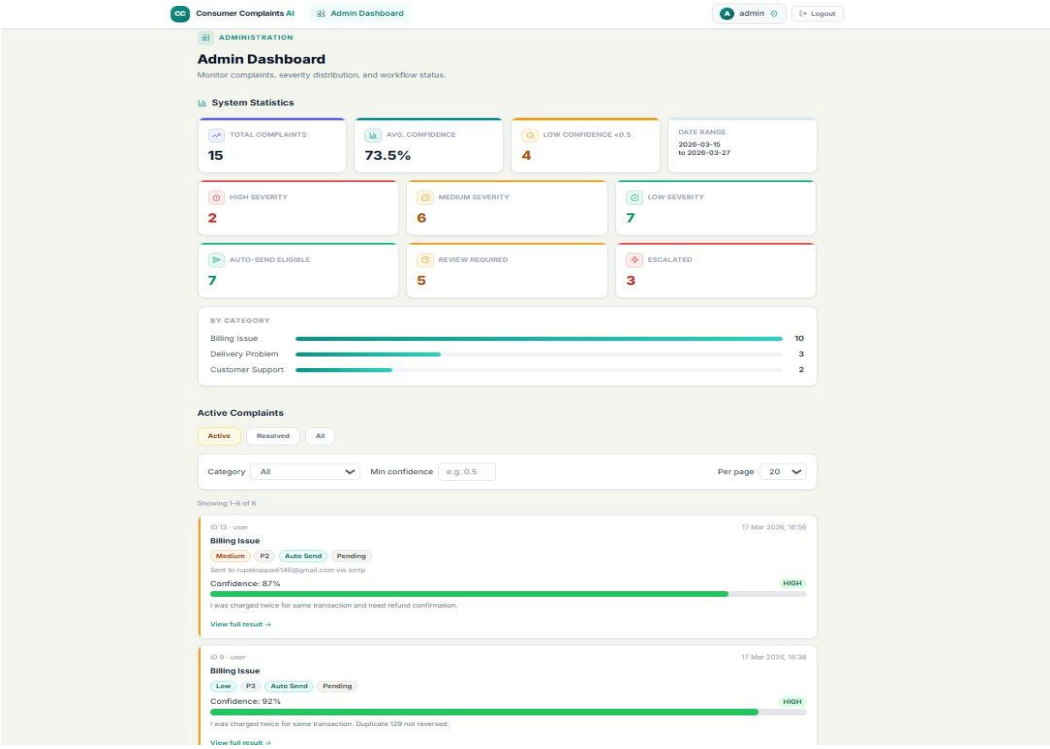


Fig. 6.7: Admin Dashboard — Complaint List with Severity, Priority, and Routing Tags

This detailed tabular view enables complaint triage and assignment. Severity badges, priority cues, and routing labels provide decision support so administrators can quickly identify high-impact cases and route them to the appropriate teams.

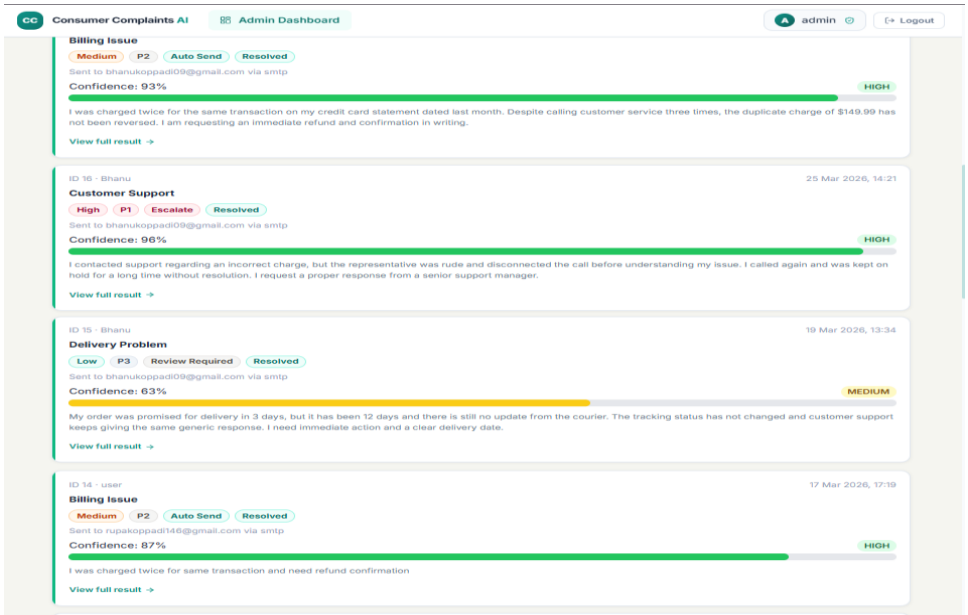
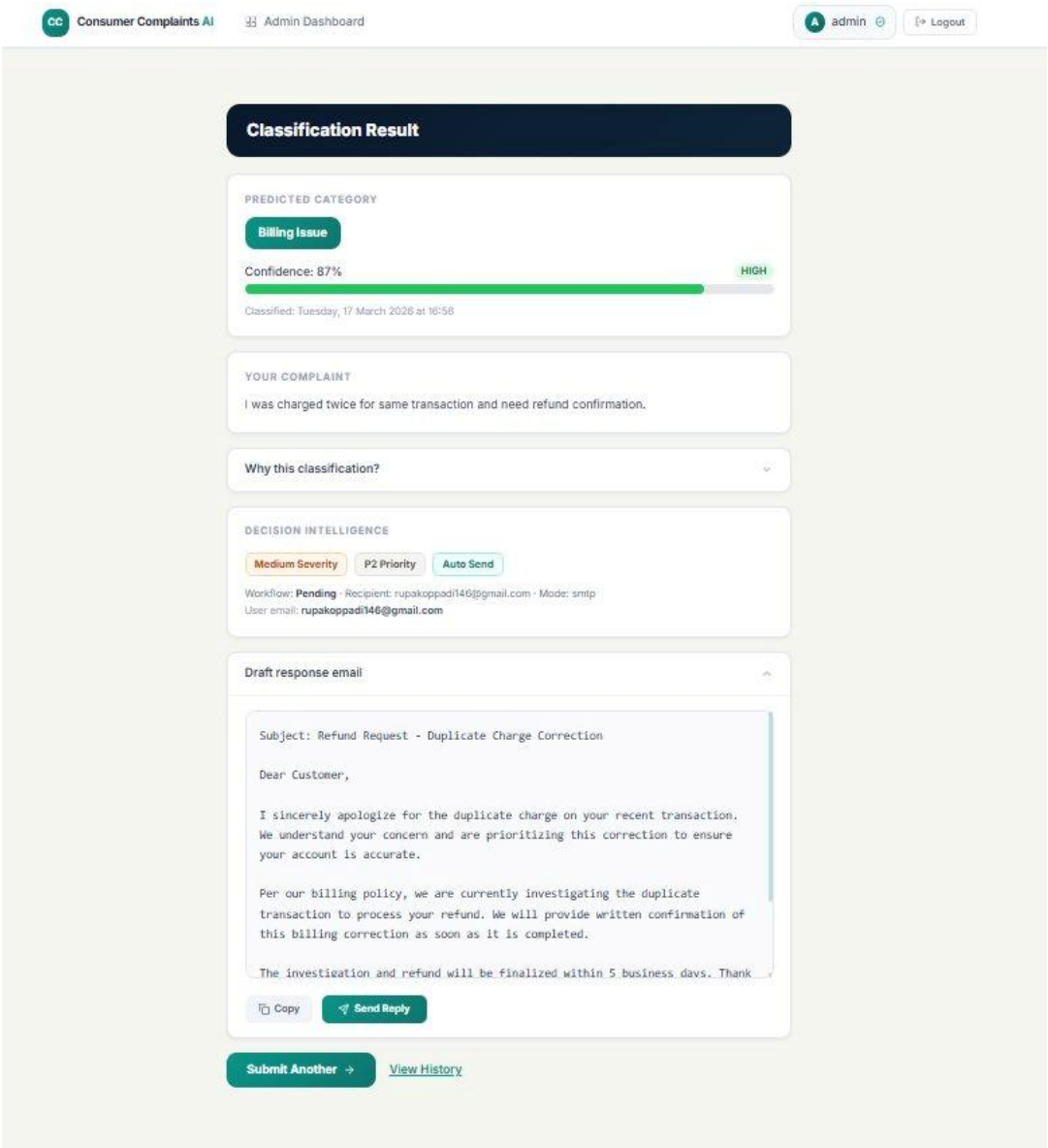


Fig. 6.8: Classification Result — Predicted Category, Confidence, Severity Badge, Decision Intelligence, and GenAI Draft Email

The result screen combines AI outputs into a single actionable view. It includes predicted category, confidence score, severity classification, rule-based decision signals, and a GenAI-generated draft response, reducing manual effort and improving response consistency.


Consumer Complaints AI
Admin Dashboard
admin
Logout
Classification Result
PREDICTED CATEGORY
Billing Issue
Confidence: 87%
HIGH
Classified: Tuesday, 17 March 2026 at 16:58
YOUR COMPLAINT
I was charged twice for same transaction and need refund confirmation.
Why this classification?
DECISION INTELLIGENCE
Medium Severity
P2 Priority
Auto Send
Workflow: Pending · Recipient: rupakoppadi146@gmail.com · Mode: smtp
User email: rupakoppadi146@gmail.com
Draft response email
Subject: Refund Request - Duplicate Charge Correction
Dear Customer,
I sincerely apologize for the duplicate charge on your recent transaction. We understand your concern and are prioritizing this correction to ensure your account is accurate.
Per our billing policy, we are currently investigating the duplicate transaction to process your refund. We will provide written confirmation of this billing correction as soon as it is completed.
The investigation and refund will be finalized within 5 business days. Thank
Copy
Send Reply
Submit Another
View History

Chapter 7: Conclusion & Future Scope

7.1 Summary of Work

This project successfully delivered a production-ready, full-stack AI-driven consumer complaint intelligence system that goes significantly beyond simple automated classification. The following key deliverables were completed:

- A complete Five-Layer Enterprise N-Tier Architecture with strict separation of concerns across Presentation, Application, Hybrid AI/Intelligence, Data Access, and Database tiers.
- A Predictive ML Pipeline (TF-IDF + Logistic Regression via Scikit-learn) achieving 100% classification accuracy on the balanced training dataset.
- A Rule-Based Severity Assessment Engine enabling zero-latency P1/P2/P3 prioritisation — ensuring no emergency complaint is buried in a standard queue.
- A Decision Engine Matrix combining ML confidence and severity to produce explicit, auditable routing decisions (Auto-Send, Review, Escalate).
- A GenAI Auto-Email Dispatcher using Google Gemini to draft and autonomously send resolution emails for eligible complaints.
- A secure Flask REST API with JWT authentication, RBAC, rate limiting, and OpenAPI/Swagger documentation.
- A React 19.2 SPA with an interactive intelligence dashboard, complaint submission, history tracking, and admin analytics.
- A full IEEE-format research paper with 13 sections, 3 original architecture diagrams, and 14 peer-reviewed references.

7.2 Limitations

- Data Dependency: The ML classifier's accuracy is bounded by the volume and label quality of training data. Class imbalance in less-common complaint categories can reduce minority-class precision in real-world deployments.
- English Only: NLP preprocessing and severity keyword heuristics are calibrated exclusively for English-language input.
- External API Dependency: The GenAI explanation and email drafting modules depend on Google Gemini API availability, latency, and quota limits.
- Keyword Coverage: The deterministic severity detector's coverage is limited to explicitly defined keyword sets and may miss novel phrasing not present in the initial vocabulary.

7.3 Future Enhancements

- Multilingual Intelligence: Integrate multilingual transformer embeddings (mBERT, XLM-R) to support complaint prioritisation across language boundaries.
- Deep Learning Substitution: Replace the TF-IDF/Logistic Regression pipeline with fine-tuned domain-specific LLM encoders (e.g., FinBERT for financial complaints).
- Sentiment & Emotion Intelligence: Layer deep sentiment analysis over the severity assessment to produce granular emotional intelligence scores.
- Full Automated Redressal: Extend the GenAI pipeline to handle the complete complaint lifecycle autonomously, including follow-up messages and closure confirmation.
- Real-Time Stream Processing: Integrate Apache Kafka or Redis Streams for real-time complaint ingestion from multiple channels simultaneously.
- Voice Input Processing: Add a speech-to-text module to accept audio complaint submissions.

7.4 Sustainability & Societal Impact

Environmental Sustainability

The system's open-source stack requires minimal computational resources compared to commercial AI platforms, reducing energy costs and carbon footprint. The deterministic Severity Assessment module operates at near-zero latency without GPU requirements, making the system energy-efficient.

Societal Impact

- Consumer Empowerment: Citizens gain transparency about how their complaints are processed through XAI rationale, building trust in automated systems.
- Economic Efficiency: Autonomous resolution of low-severity complaints significantly reduces operational costs, allowing human agents to focus on genuine emergencies.
- Equity of Priority: The deterministic P1 detection ensures that vulnerable consumers reporting fraud, injury, or legal threats are never deprioritised due to queue backlog.
- Digital Inclusion: The domain-agnostic design makes enterprise AI accessible to smaller organisations through its zero-licensing-cost open-source stack.
- Academic Contribution: The IEEE research paper documents the hybrid methodology for the broader research community.

References

[1] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[2] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. O'Reilly Media, 2018.

[3] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," [Online]. Available: https://www.postgresql.org/docs/16/ [Accessed: Mar. 2026].

[4] T. B. Brown et al., "Language Models are Few-Shot Learners," Adv. Neural Inf. Process. Syst., vol. 33, pp. 1877-1901, 2020.

[5] G. Banga and S. Peddireddy, "AI-Driven Customer Complaint Management in CRM Systems," International Journal of Advanced Research in Computer Science, vol. 14, no. 2, 2023.

[6] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why Should I Trust You?': Explaining the Predictions of Any Classifier," in Proc. ACM SIGKDD, pp. 1135-1144, 2016.

[7] O. Aydin and E. Karaarslan, "Complaint Detection and Classification of Customer Reviews," in Proc. 2021 5th Int. Symp. ISMSIT, Ankara, Turkey, 2021.

[8] J. Alegado, J. Bernabe, A. Go, and C. Ponio, "Automated Customer Complaint Routing Using Natural Language Processing," IJIREEICE, vol. 10, no. 5, pp. 88-94, 2022.

[9] D. Shetty, A. Kumar, and S. R. Prasad, "Machine Learning-Based Classification of Customer Complaints in Banking Sector," Int. J. Eng. Research & Technology, vol. 11, no. 7, pp. 352-358, 2022.

[10] A. Das, S. Jana, and P. Majumder, "TF-IDF Based Text Feature Extraction for Consumer Complaint Analysis," Int. J. Advanced Computer Science and Applications, vol. 14, no. 3, 2023.

[11] Y. Huang, W. Tian, Z. Liu, and Q. Mei, "Complaint and Severity Identification from Online Financial Content," IEEE Trans. Computational Social Systems, vol. 10, no. 4, pp. 1720-1731, Aug. 2023.

[12] S. Ramadhana et al., "Deep Learning and NLP for Enhanced Customer Complaint Automation in Finance," IJIRSET, vol. 13, no. 8, pp. 9410-9420, 2024.

[13] B. Mehic and A. Adilovic, "Security Configuration for Microservices Using Spring Security with OAuth2," IEEE Access, vol. 10, pp. 58420-58432, 2022.

[14] C. Shen et al., "Machine Learning and NLP Approaches for Consumer Complaint Classification in Healthcare Systems," Int. J. Medical Informatics, vol. 185, 2025.

[15] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, pp. 4171-4186, 2019.

[16] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[17] E. Brynjolfsson, D. Li, and L. R. Raymond, "Generative AI at Work," NBER Working Paper, no. 31161, 2023.

[18] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint arXiv:2312.11805, 2023.

[19] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv preprint arXiv:1907.11692, 2019.

[20] A. Holzinger et al., "Causability and Explainability of Artificial Intelligence in Medicine," WIREs Data Min. Knowl. Discov., vol. 9, no. 4, 2019.

Appendices

Appendix A: Source Code Repository Structure

```
consumer-complaints-ml-genai-ntier/
├── docs/
│   ├── IEEE_Consumer_complaints.docx
│   └── api/openapi.yaml
├── infra/
│   └── docker-compose.yml
├── src/
│   ├── backend/
│   │   ├── app.py                # Flask REST API
│   │   ├── config.py
│   │   ├── database/db.py        # SQLAlchemy ORM
│   │   ├── ml/
│   │   │   ├── predictor.py        # TF-IDF + Logistic Regression
│   │   │   ├── severity_detector.py  # Rule-based severity engine
│   │   │   └── decision_engine.py    # Routing decision matrix
│   │   ├── genai/
│   │   │   ├── explanation.py       # Gemini explanation generator
│   │   │   ├── response_drafter.py   # Email response drafter
│   │   │   └── email_sender.py       # Auto-email dispatcher
│   │   ├── preprocessing/
│   │   └── tests/                # pytest test suite
│   └── frontend/
│       └── src/
│           ├── pages/
│           ├── components/
│           ├── store/
│           └── api/
```

Appendix B: API Endpoint Reference

Method	Endpoint	Auth	Description
GET	/	None	Health check
GET	/health	None	System status
GET	/api/docs	None	Swagger UI
POST	/api/auth/register	None	User registration
POST	/api/auth/login	None	JWT login
POST	/api/auth/refresh	Refresh JWT	Token refresh

POST	/api/classify	Optional JWT	Classify complaint
POST	/api/complaints	JWT (USER)	Submit complaint
GET	/api/complaints	JWT (USER)	Get user's complaints
GET	/api/complaints/<id>	JWT	Get single complaint
GET	/api/explanation/<id>	Optional JWT	Get explanation
POST	/api/draft-response	JWT	Draft email response
GET	/api/admin/complaints	JWT (ADMIN)	Admin complaint list
GET	/api/admin/statistics	JWT (ADMIN)	System statistics
POST	/api/admin/complaints/<id>/reply	JWT (ADMIN)	Send admin reply

Appendix C: Environment Configuration (.env)

```
# Backend environment configuration
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/consumer_complaints
GEMINI_API_KEY=
GEMINI_MODEL=gemini-1.5-flash
ML_MODEL_PATH=models/complaint_classifier.pkl
VECTORIZER_PATH=models/tfidf_vectorizer.pkl
USE_TRANSFORMER_MODEL=false
SECRET_KEY=change-me-in-production
JWT_SECRET_KEY=change-me-jwt-in-production
CREATE_DEFAULT_USERS=false
DEFAULT_ADMIN_USERNAME=admin
DEFAULT_ADMIN_PASSWORD=Admin@123
ALLOWED_ORIGINS=http://localhost:5173
FLASK_APP=app.py
FLASK_DEBUG=0
EMAIL_DELIVERY_MODE=simulate
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USE_TLS=true
```